Automate database tasks for Azure SQL

- Scripting repetitive tasks in Azure SQL for regular maintenance jobs and multi-instance administration.
- Automation of error and resource alerts.
- Configuration of notifications for failures.

Prerequisites

- Ability to use tools for running queries against a Microsoft SQL database, either on-premises on cloud-based
- Ability to write code in the SQL language, particularly the Microsoft T-SQL dialect, at a basic level.
- Experience creating and configuring resources using the Azure portal

Automate database deployment

Intermediate

Data Analyst

Data Engineer

Database Administrator

Azure

- **Azure SQL Database**
- **SQL Server**
- Explore multiple deployment models available on Azure.

  Use Bicep files and Azure Resource Manager (ARM) templates for deploying Azure SQL resources.
- Understand how to use PowerShell and Azure CLI for automation purposes.

Learning objectives

- After completing this module, you will be able to:
- Describe the deployment models available on Azure
- Deploy database resources using PowerShell and Azure CLI
- Deploy Azure SQL resources using Bicep files and Azure Resource Manager templates
- Understand the difference between multiple command-line options

Prerequisites

- Ability to use tools for running queries against a Microsoft SQL database, either on-premises on cloud-based.
- Ability to write code in the SQL language, particularly the Microsoft T-SQL dialect, at a basic level.
- Experience creating and configuring resources using the Azure portal.

Introduction

- **1 minute**
- On-premises systems require cabling and racks of hardware in order to deploy a new database server.

  With cloud computing, this isn't necessary.
- One of the major benefits of cloud computing is that system resources are abstracted behind an API.
- A common term in cloud computing deployments is infrastructure as code.
- This means all your resources are defined using scripts stored in source control, making it easy to deploy them to new environments.
- While stateful resources don't need to be deployed as often as application code, defining your infrastructure ensures consistent implementation, reducing configuration risks and human errors.
- Whether you're managing a single database or a complex multi-database setup, this module equips you with the skills to automate deployments efficiently and effectively.

Learning objectives

- After completing this module, you'll be able to:
- Describe the deployment models available on Azure

- Deploy database resources using PowerShell and Azure CLI
- Deploy Azure SQL resources using Bicep files and Azure Resource Manager templates
- Understand the difference between multiple command-line options

Describe deployment models in Azure

- Azure Resource Manager (ARM) templates and Azure Bicep files allow you to deploy a complete set of resources using a single declarative template.
- Bicep is Microsoft's recommended authoring language, and Bicep files compile into ARM JSON templates at deployment time.
- You can define dependencies within these templates and use parameters to adjust deployment values at runtime.
- Once you have a template, you can deploy it in several ways, including through Azure Pipelines or the custom deployment options in the Azure portal.
- These deployments use a declarative model, specifying what should be created, while the ARM template framework determines how to deploy it.
- In contrast, the imperative model, used by tools like PowerShell and Azure CLI, follows a specific sequence of tasks to be executed.

Azure Resource Manager templates and Bicep

Azure Resource Manager templates and Bicep files enable you to create and deploy an entire infrastructure in a declarative manner.

- For instance, you can deploy a virtual machine along with its network and storage dependencies in one document.
- Both formats support orchestration, managing the deployment of interdependent resources in the correct order.
- They also support extensibility, allowing you to run PowerShell or Bash scripts on your resources post-deployment.
- Because Bicep files compile into ARM templates, the two formats share the same underlying capabilities — Bicep just offers a more concise authoring experience.

PowerShell

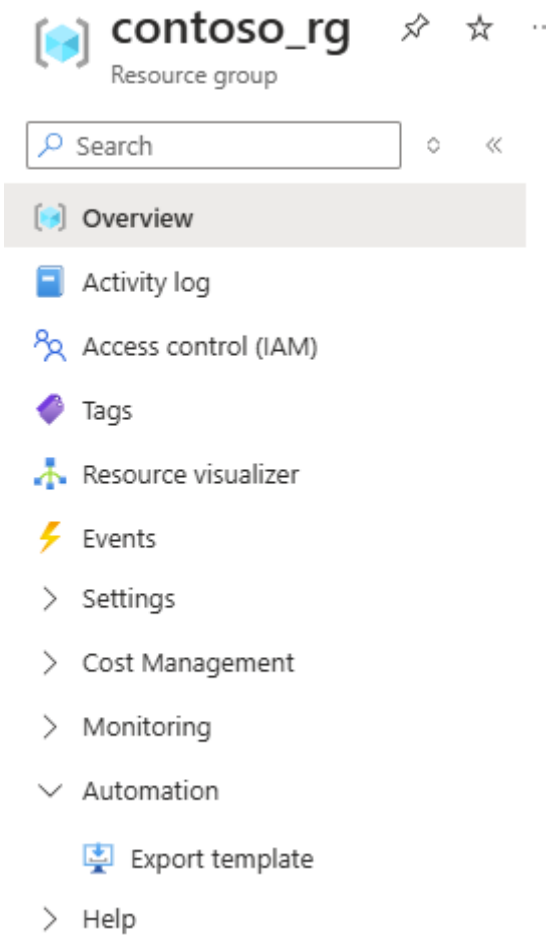
- PowerShell, with its Az module, provides cmdlets for nearly all Azure services.
- For example, Az. Compute covers Azure Virtual Machines.
- PowerShell is often used for resource modification and status retrieval.
- While it can create resources, it isn't typically used for complex deployments.
- PowerShell can deploy both ARM templates and Bicep files, supporting both declarative and imperative models.

Azure CLI

- Azure Command-Line Interface (CLI) is similar to PowerShell, supporting both imperative and declarative operations.
- The Azure CLI can deploy or modify Azure resources, and some commands for Azure PostgreSQL and Azure MySQL Databases are exclusive to the CLI.

- Azure portal
- The Azure portal is a graphical interface for ARM templates.

Any resources you create and deploy using the portal have an ARM template that you can export by selecting Export template in the Automation section of your resource group.
From the exported template, you can also generate the equivalent Bicep file by using the Bicep decompile command.



- The Azure portal is often the easiest way to get started with Azure.
- As organizations and DBAs gain experience, they typically move to more automated deployment models.
- Azure DevOps Services
- In Azure DevOps Services, deployments are managed using Azure Pipelines, a comprehensive CI/CD service that automates the build, testing, and deployment of your code.
- You can deploy Azure resources using Bicep files or ARM templates in two ways: by calling a PowerShell script or by defining tasks that stage your artifacts (templates and required secrets) and then deploy the templates.
- This automation ensures that your deployments are consistent and reliable, reducing the risk of human error.
- Continuous integration (CI) complements this by focusing on making small, frequent changes to code and regularly checking it into a version control system.
- CI automates the build, packaging, and testing of applications, facilitating better collaboration among developers and improving code quality.
- Continuous delivery (CD) builds on CI by automating the delivery of code changes to the underlying infrastructure, ensuring that your deployments aren't only consistent but also rapid and efficient.
- Together, Azure Pipelines and CI/CD practices streamline the entire development and deployment process, making it more efficient and reliable.

- Automate deployment by using Azure Resource Manager templates and Bicep
- Automating database deployment is a crucial ability to create a reliable and sustainable development process.
- This unit provides you with the information to be able to use Bicep files and Azure Resource Manager (ARM) templates in your database deployments.

Bicep

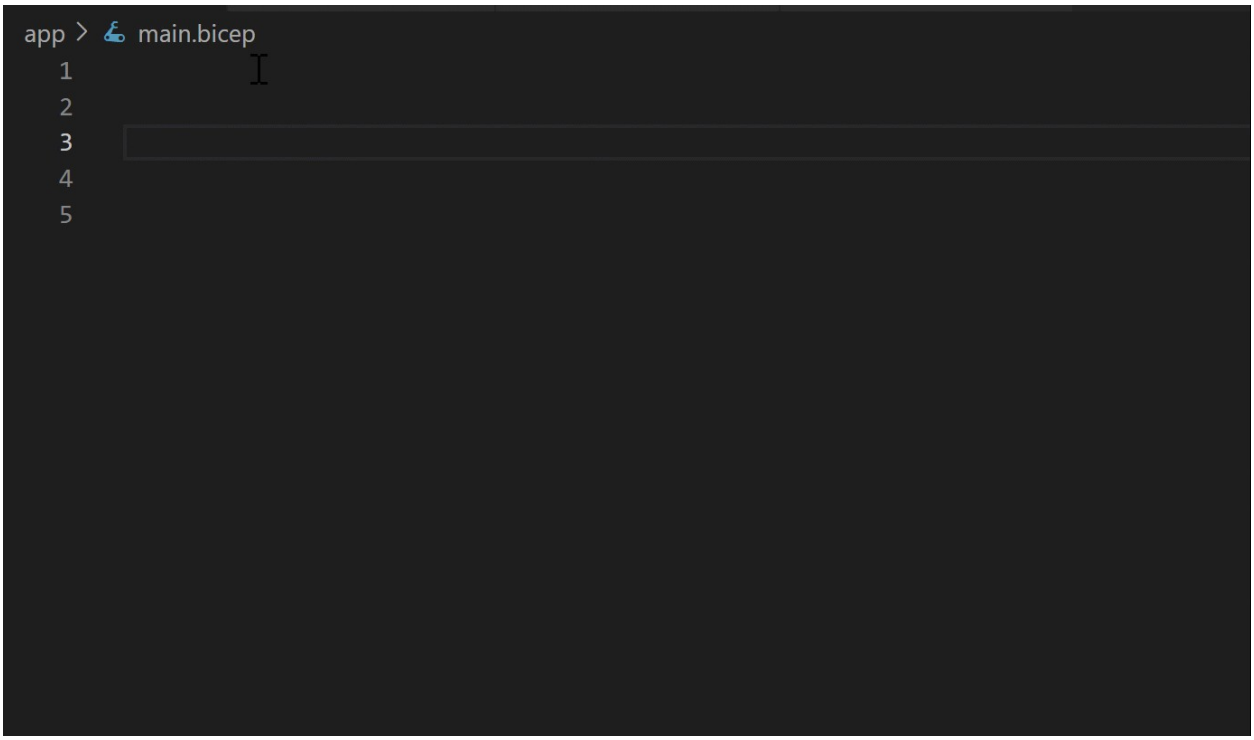
- Azure Bicep is Microsoft's recommended declarative language for deploying Azure resources.
- A Bicep file compiles into an Azure Resource Manager (ARM) JSON template at deployment time, so a Bicep-based workflow gives you all the capabilities of ARM templates with a more concise syntax that's easier to author and maintain.

- Bicep isn't intended to be a general-purpose programming language.
- Instead, it's a specialized tool for creating files that declare Azure infrastructure resources and their properties.
- This approach ensures consistent resource deployment throughout the development lifecycle.

Benefits

Bicep offers the following benefits:

- Continuous full support – Bicep provides support for all resource types and API versions for Azure services, which means that as soon as a resource provider introduces new resource types and API versions, you can use them in your Bicep file without waiting for a tool update.
- Simple syntax – Compared to an equivalent JSON file, Bicep files are more concise and easier to read.
- Easy to use – Bicep requires no previous knowledge of programming languages and is easy to write and understand.



- Idempotent deployments – Like ARM templates, Bicep deployments are idempotent, so you can repeatedly deploy your infrastructure throughout the development lifecycle with confidence that your resources end up in a consistent state.
- Modular and reusable – Bicep files can be split into modules and combined, so you can compose the deployments you need and share common patterns across projects.
- Authoring tools – You can author Bicep files in Visual Studio Code with the Bicep extension, which provides IntelliSense, syntax validation, and in-line help.

- Deploy a Bicep file with PowerShell
- You have several options for the scope of your deployment when using PowerShell and Bicep.
- You can deploy to a resource group, a subscription, a management group (a collection of subscriptions under the same Azure tenant, commonly used in large enterprise deployments), or a tenant.

Here's a Bicep definition to create a single database in Azure SQL Database:

Bicep

```
@description('The name of the SQL logical server.')
param serverName string = uniqueString('sql', resourceGroup().id)


@description('The name of the SQL Database.')
param sqlDBName string = 'SampleDB'


@description('Location for all resources.')
param location string = resourceGroup().location
```

```
@description('The administrator username of the SQL logical server.')
param administratorLogin string

@description('The administrator password of the SQL logical server.')
@secure()
param administratorLoginPassword string

resource sqlServer 'Microsoft. Sql/servers@2022-02-01' = {
name: serverName
location: location
properties: {
administratorLogin: administratorLogin
administratorLoginPassword: administratorLoginPassword
}
}

resource sqlDB 'Microsoft. Sql/servers/databases@2022-02-01' = {
parent: sqlServer
name: sqlDBName
location: location
sku: {
name: 'Standard'
tier: 'Standard'
}
}
```

- In this example, a single database is defined using the DTU-based Standard tier.
- When creating the database, you also specify the server that manages it and the Azure region where it's located.
- To deploy this file, save it as main.bicep on your local computer and run the following commands in PowerShell:

PowerShell

```
$projectName = Read-Host -Prompt "Enter a project name that is used for generating resource names"
$location = Read-Host -Prompt "Enter an Azure location (e.g., centralus)"
$adminUser = Read-Host -Prompt "Enter the SQL Server administrator username"
$adminPassword = Read-Host -Prompt "Enter the SQL Server administrator password" -AsSecureString

$resourceGroupName = "${projectName}rg"

# Create a new resource group
New-AzResourceGroup -Name $resourceGroupName -Location $location

# Deploy resources using a Bicep file
New-AzResourceGroupDeployment -ResourceGroupName $resourceGroupName -TemplateFile ./main.bicep
-administratorLogin $adminUser -administratorLoginPassword $adminPassword
```

- This script prompts the user for a project name, Azure location, and SQL Server administrator credentials, creates a new resource group in the specified location, and deploys the Bicep file that provisions the Azure SQL logical server and database.
- Azure Resource Manager (ARM) templates
- Azure Resource Manager (ARM) templates are JavaScript Object Notation (JSON) documents that describe the resources to deploy within an Azure Resource Group.

- Bicep files compile into this JSON format at deployment time, so understanding ARM templates is useful when you're reading exported templates, working with existing solutions, or troubleshooting deployments.
- ARM templates are declarative, allowing you to specify your resources and properties without writing a full sequence of programming commands.
- They support orchestration (managing the deployment of interdependent resources in the correct order) and extensibility (allowing you to run PowerShell or Bash scripts after your resources are deployed).

  You can also export the ARM template of any existing resource group from the Azure portal, which is a useful way to learn template syntax or reproduce an existing environment.

Bicep vs. JSON

- The following examples show the difference between a Bicep file and the equivalent JSON template.
- Both examples deploy a storage account.

JSON

```
{
"$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.json#",
"contentVersion": "1.0.0.0",
"parameters": {
"location": {
"type": "string",
"defaultValue": "[resourceGroup().location]"
},
"storageAccountName": {
"type": "string",
"defaultValue": "[format('toylaunch{0}', uniqueString(resourceGroup().id))]"
}
},
"resources": [
{
"type": "Microsoft. Storage/storageAccounts",
"apiVersion": "2022-09-01",
"name": "[parameters('storageAccountName')]",
"location": "[parameters('location')]",
"sku": {
"name": "Standard_LRS"
},
"kind": "StorageV2",
"properties": {
"accessTier": "Hot"
}
}
]
}
```

Bicep

```
param location string = resourceGroup().location
param storageAccountName string = 'toylaunch${uniqueString(resourceGroup().id)}'

resource storageAccount 'Microsoft. Storage/storageAccounts@2022-09-01' = {
```

```
name: storageAccountName
location: location
sku: {
name: 'Standard_LRS'
  }
kind: 'StorageV2'
properties: {
accessTier: 'Hot'
  }
}
```

- The Bicep file is shorter and easier to read while producing the same deployment.
- You can also install the Bicep extension for Visual Studio Code to author Bicep files with rich IntelliSense and syntax validation.

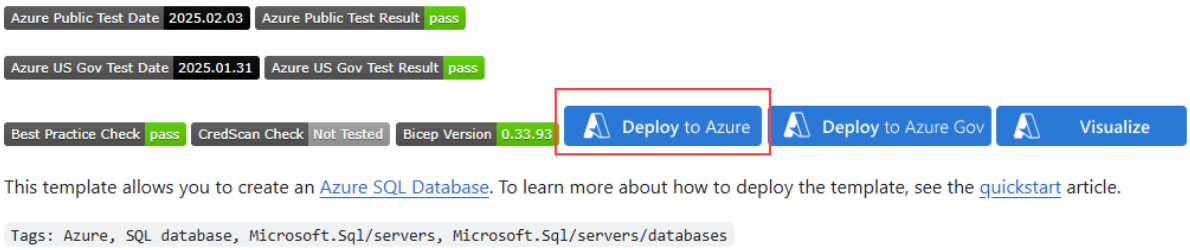
- Source control for templates
- Bicep files and ARM templates exemplify infrastructure as code.

  With hardware resources abstracted behind APIs, your entire infrastructure becomes an integral part of your application code.
- Just like application or database code, it's crucial to protect and version this code.
- Besides the internal versioning within a template, your source control system should also version your Bicep files and ARM templates.

  Typically, database administrators won't create Bicep files or ARM templates from scratch.
  Instead, they can build them using the Azure portal or use templates from the quickstart templates provided by Microsoft on GitHub.

description	page_type	products		urlFragment	languages	
This template allows you to create SQL Database and Server.	sample	azure	azure-resource-manager	sql-database	json	bicep

Create a SQL Server and Database



When you select "Deploy to Azure" on the GitHub page for SQL Database templates, it takes you to the Azure portal.

- The template loads up, and you just need to fill in a few details like resource group, location, and admin credentials.
- After that, select Review + create and then Create to start the deployment.
- The portal handles the rest and show you the status until it's done.

Automate deployment by using PowerShell

- PowerShell is a modern, cross-platform command shell designed to simplify task management and enhance automation.
- It provides administrators with powerful command-line features that, when automated, help reduce operational costs.
- PowerShell can handle both text and .NET objects, making it a versatile, all-in-one command-line tool.

- Some key benefits of PowerShell include:
- **Robust command-line history**
- **Tab completion and command prediction**
- **Support for command and parameter aliases**
- **Pipeline for chaining commands**
- **In-console help system**
- PowerShell's core module, the Az PowerShell module, is an open-source set of cmdlets.

  It allows you to manage Azure resources directly from PowerShell, enabling resource creation, modification, status retrieval, and template-based deployments.

Az. Sql PowerShell module

- The Az. Sql PowerShell module is a subset of the Az PowerShell module, enabling you to manage and deploy Azure SQL resources.

  With Az. Sql cmdlets, you can handle everything from creating databases to configuring geo-replication and full Azure SQL management.
- You can use the Az. Sql PowerShell module in various environments, including PowerShellGet, Azure Cloud Shell, and an Az PowerShell Docker container.
- No matter how you use PowerShell, the syntax remains consistent with the verb-noun structure.
- Powershell
- <command-name> -<Required Parameter Name> <Required Parameter Value>

  [-<Optional Parameter Name> <Optional Parameter Value>]
  [-<Optional Switch Parameters>]
  [-<Optional Parameter Name>] <Required Parameter Value>
- Commands always begin with a command name, such as Get-AzSqlServer, which returns information about one or more logical servers for Azure SQL Database.
- The "command-name" is then followed by a Parameter Name with <-ServerName> being an applicable parameter for Get-AzSQLServer.
- This is then followed with a parameter value, which is written in a string form.
- The following example shows the usage of the Get-AzSqlServer command with multiple parameters with its return values:

PowerShell

```
Get-AzSqlServer -ResourceGroupName "ResourceGroup01" -ServerName "Server01"
```

Here are a few more examples, such as how to create a new SQL Managed Instance and how to create a database on a specific server:

Powershell

```
New-AzSqlInstance -Name managedInstance2 -ResourceGroupName ResourceGroup01 -Location westcentralus
-AdministratorCredential (Get-Credential) -SubnetId
"/subscriptions/xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx/resourceGroups/resourcegroup01/providers/Microsoft.
Network/virtualNetworks/vnet_name/subnets/subnet_name" -LicenseType LicenseIncluded -StorageSizeInGB 1024 -VCore
16 -Edition "GeneralPurpose" -ComputeGeneration Gen5
```

Powershell

```
New-AzSqlDatabase -ResourceGroupName "ResourceGroup01" -ServerName "Server01" -DatabaseName "Database01"
```

- Here's an example that creates a new SQL Managed Instance with External Microsoft Entra administrator, Microsoft Entra-only authentication, and no SqlAdministratorCredentials:

Powershell

```
New-AzSqlInstance -Name managedInstance2 -ResourceGroupName ResourceGroup01 -ExternalAdminName DummyLogin
-EnableActiveDirectoryOnlyAuthentication -Location westcentralus -SubnetId "/subscriptions/xxxxxxxx-xxxx-xxxx-xxxx-
xxxxxxxxxxxx/resourceGroups/resourcegroup01/providers/Microsoft.
Network/virtualNetworks/vnet_name/subnets/subnet_name" -LicenseType LicenseIncluded -StorageSizeInGB 1024 -VCore
16 -Edition "GeneralPurpose" -ComputeGeneration Gen5
```

```
$val = Get-AzSqlInstance -Name managedInstance2 -ResourceGroupName ResourceGroup01
-ExpandActiveDirectoryAdministrator
```

- To learn more about the full list of command-names for the Az. Sql module, see Azure PowerShell Az. Sql.

Automate deployment by using Azure CLI

- Database automation is no longer just for large enterprises; it's now essential for businesses of all sizes to stay competitive.
- As a database administrator, automating database tasks is crucial for several reasons:
- Granular control over applications and databases
- Easy scalability, enhancing efficiency when managing numerous assets
- Reusability of scripts for automating routine tasks
- Simplified troubleshooting when GUI tools are unavailable
- The Azure Command-Line Interface (CLI) is a cross-platform tool that helps you create and manage Azure resources.
- You can run commands through the terminal using interactive prompts or scripts.
- Azure CLI can be installed on Linux, Mac, or Windows computers.
- You can also run it from a browser using the Cloud Shell terminal on the Azure portal or inside a Docker container.
- The Azure CLI syntax follows the reference name - command - parameter - parameter value pattern.
- For example, switching between subscriptions is a common task.
- Here's the syntax:

Azure CLI

```
az account set --subscription "my subscription name"
```

PowerShell vs. Azure CLI

Azure PowerShell and Azure CLI are both cross-platform command-line tools that allow you to create and manage Azure resources on Windows, macOS, and Linux.

- The primary difference between them lies in the shell environments they support.

Shell Environment	Azure CLI	Azure Powershell
Cmd	Yes	
Bash	Yes	
Windows PowerShell	Yes	Yes
PowerShell	Yes	Yes

- To choose the right tool, consider your experience and your work environment.
- Azure CLI is similar to bash scripting, making it intuitive for those who typically work with Linux systems.
- On the other hand, Azure PowerShell includes modules that help manage Azure resources from PowerShell.
- PowerShell commands follow the standard verb-noun syntax, making it a natural fit for those familiar with Windows systems.
- Here's a quick comparison of some commonly used commands in both their CLI and PowerShell forms:

Command	Azure CLI	Azure PowerShell
Sign in with Web Browser	az login	Connect-AzAccount
Get available subscriptions	az account list	Get-AzSubscription
Set Subscription	az account set –subscription	Set-AzContext -Subscription
List all virtual machines	az vm list	Get-AzVM
Create a new server	az sql server create	New-AzSqlServer

Deploying SQL Database using Azure CLI

Here's an example of how to deploy an SQL Database and create a firewall rule to allow access from Azure services using Azure CLI:

Azure CLI

```
let "randomIdentifier=$RANDOM*$RANDOM"
```

```
$resourceGroup = "<your resource group>"
$location = "<your location preference>"
$server = "dp300-sql-server-$randomIdentifier"
$login = "sqladmin"
$password = "Pa$$w0rD-$randomIdentifier"
```

```
az sql server create --name $server --resource-group $resourceGroup --location "$location" --admin-user $login --admin-password $password
```

```
az sql server firewall-rule create --resource-group $resourceGroup --server $server -n AllowYourIp --start-ip-address 0.0.0.0 --end-ip-address 0.0.0.0
```

- To learn more about all the Azure SQL CLI commands available, see Azure SQL CLI commands.
- Deploying Bicep files and ARM templates using Azure CLI and PowerShell

With PowerShell, you have multiple options for the scope of your deployment.

- You can deploy to a resource group, a subscription, a management group (a collection of subscriptions under the same Azure tenant, commonly used in large enterprise deployments), or a tenant.
- Bicep files and Azure Resource Manager templates are parameterized, requiring you to pass in parameters either inline or through a parameter file, as shown in the following example.

PowerShell

```
New-AzResourceGroupDeployment -Name ExampleDeployment -ResourceGroupName ExampleResourceGroup `
-TemplateFile c:\MyTemplates\main.bicep `
-TemplateParameterFile c:\MyTemplates\storage.parameters.json
```

- The same -TemplateFile parameter accepts either a Bicep file (.bicep) or an ARM template (.json).
- The parameter and template files can be stored in a Git repository, Azure Blob Storage, or any other accessible location from the deploying machine.
- Azure CLI offers the same deployment scope options as PowerShell.
- You can use local or remote parameter files and templates, just as you would with PowerShell, as shown in the following example.

Azure CLI

```
az deployment group create --resource-group ExampleResourceGroup --template-file '\path\main.bicep'
```

- To deploy remote linked templates with relative path that are stored in a storage account, use query-string to specify the SAS token:

Azure CLI

```
az deployment group create \
--name linkedTemplateWithRelativePath \
--resource-group myResourceGroup \
--template-uri "https://stage20210126.blob.core.windows.net/template-staging/mainTemplate.json" \
--query-string $sasToken
```

Note

Azure CLI supports deploying local Bicep files directly with az deployment group create --template-file main.bicep, but doesn't support deploying remote Bicep files directly by URI.
To deploy a remote Bicep file, use the Bicep CLI to compile it into a JSON template first, and then deploy the JSON template from a remote location.

Exercise: Deploy an Azure SQL Database using an Azure Resource Manager template

In this exercise, you'll learn how to deploy an Azure SQL Database from a template.

Note

- To complete this exercise, you'll need an Azure subscription.
- Launch the exercise and follow the instructions.

Launch Exercise

Summary

- **1 minute**
- The Azure platform offers numerous options for deploying resources.
- You can deploy using Azure Pipelines or command line scripting.
- Azure Resource Manager templates and Bicep files give you the flexibility to deploy Azure resources in a granular, repeatable, and consistent manner.
- It's best practice to manage your resources with a source control system.
- This approach not only ensures more consistent deployments but also reduces the risk of configuration errors.
- Now that you've reviewed this module, you should be able to:
- To describe the deployment models available on Azure
- How to deploy database resources using PowerShell and CLI
- How to deploy Azure SQL resources using Bicep files and Azure Resource Manager templates
- The difference between multiple command-line options

Create and manage SQL Agent jobs

Intermediate

Data Analyst

Data Engineer

Database Administrator

Azure

- **Azure SQL Database**
- **SQL Server**
- Explore SQL automation for scheduled tasks, and automatic alerts for SQL Server and Azure SQL Managed Instance.

Learning objectives

- After completing this module, you will be able to:
- Schedule necessary maintenance activities for your databases.
- Configure notifications and alerts on SQL Server Agent jobs, and SQL Server.
- Configure alerts based on performance monitor values.

Prerequisites

- Ability to use tools for running queries against a Microsoft SQL Server database, either on-premises or on cloud.
- Ability to write code in the SQL language, particularly the Microsoft T-SQL dialect, at a basic level.
- Experience creating, and configuring resources using the Azure portal.

Introduction

- **1 minute**
- Database systems require regular maintenance, including tasks like backups and updating statistics.
- Maintenance may also involve scheduled jobs that run against a database.
- Common examples include extracting, transforming, and loading data from a transaction processing system into a data warehouse.
- In SQL Server and Azure SQL Managed Instance, the SQL Server Agent service allows you to schedule these maintenance tasks.
- Azure offers built-in resource monitoring, and you can also use the platform's options for handling and responding to events, enhancing your ability to maintain and manage your database systems effectively.
- Most SQL Server Agent features are supported in Azure SQL Managed Instance, but there are some minor T-SQL differences between SQL Server and Azure SQL Managed Instance that might affect job scripts.

Learning objectives

- At the end of this module, you'll understand:
- What maintenance activities you should perform on your databases
- How to configure notifications and alerts on SQL Server Agent jobs and SQL Server
- How to configure notifications alerts based on performance monitor values

Create a SQL Server maintenance plan

- Typical activities you can schedule for regular SQL Server maintenance include:
- **Database and transaction log backups**
- **Database consistency checks**
- **Index maintenance**
- **Statistics updates**
- It's crucial to understand the importance of backups, as well as index and statistics maintenance, for all your databases.
- Database consistency checks, also known as CHECKDB (using the command DBCC CHECKDB), are equally important because they are the only way to check an entire database for corruption.
- Depending on the size of your databases and your uptime requirements, you might perform all these activities nightly.
- However, in production systems, maintenance operations are often spread out over the week, as both index maintenance and consistency checks are very I/O intensive and typically done during weekend hours.
- Many DBAs stagger backups of large databases, performing one full backup a week and using differential and transaction log backups to manage recovery to a specific point in time.
- SQL Server offers a built-in way to manage all these tasks using Maintenance Plans.
- Maintenance Plans create a workflow of tasks to support your databases and are created as Integration Services packages, allowing you to schedule your maintenance activities.
- Additionally, many DBAs use open-source scripts for database maintenance to gain more flexibility and control over maintenance activities.

Best practices for maintenance plans

Maintenance plans not only help you perform database maintenance but also offer options to prune data from the msdb database, which serves as the data store for the SQL Server Agent.

Also, maintenance plans allow you to specify the removal of older database backups from disk.

- Removing old backup files reduces the size of your backup volume and helps manage the size of the msdb database.
- Ensure that your backup retention period is longer than your consistency check window.
- For example, if you run a consistency check weekly, you should retain sufficient backup history to recover from potential corruption detected during consistency checks.
- Note that the backup operation does not detect corruption in a database, so it is possible to have corruption within a backup file.
- Maintenance plan activities are scheduled as SQL Server Agent jobs for execution.

Create a maintenance plan

You can create a maintenance plan using SQL Server Management Studio, as shown below.

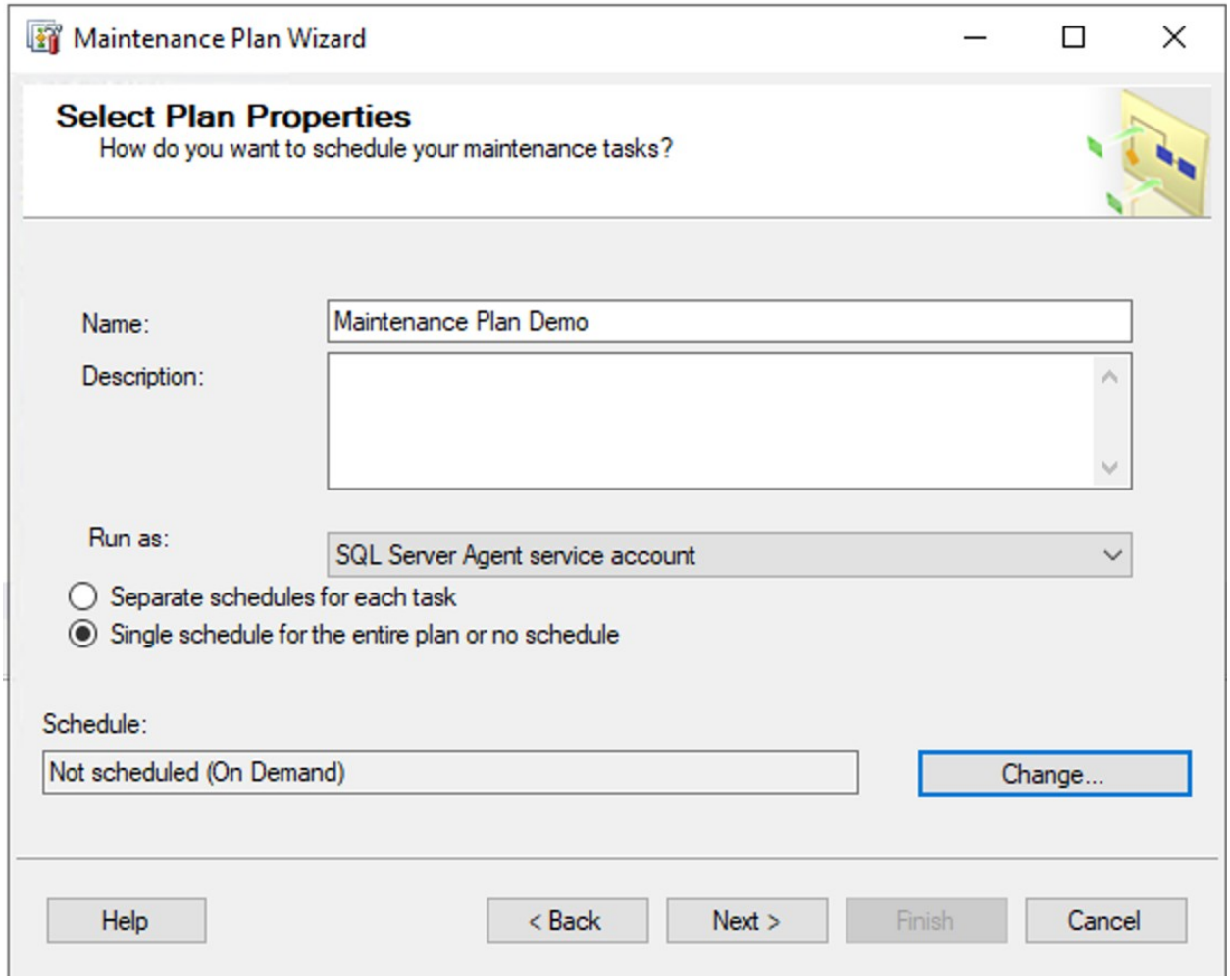
- In the example, multiple maintenance tasks are combined into one maintenance plan.

However, the best practice is to create a separate maintenance plan for each type of task, and possibly even for specific databases on your server.

For instance, you might create one maintenance plan to back up system databases and another to back up user databases.

- Additionally, you could have a separate maintenance plan for handling the backup of a particularly large user database.

The image below and the following examples demonstrate how to create a maintenance plan using the Maintenance Plan Wizard.



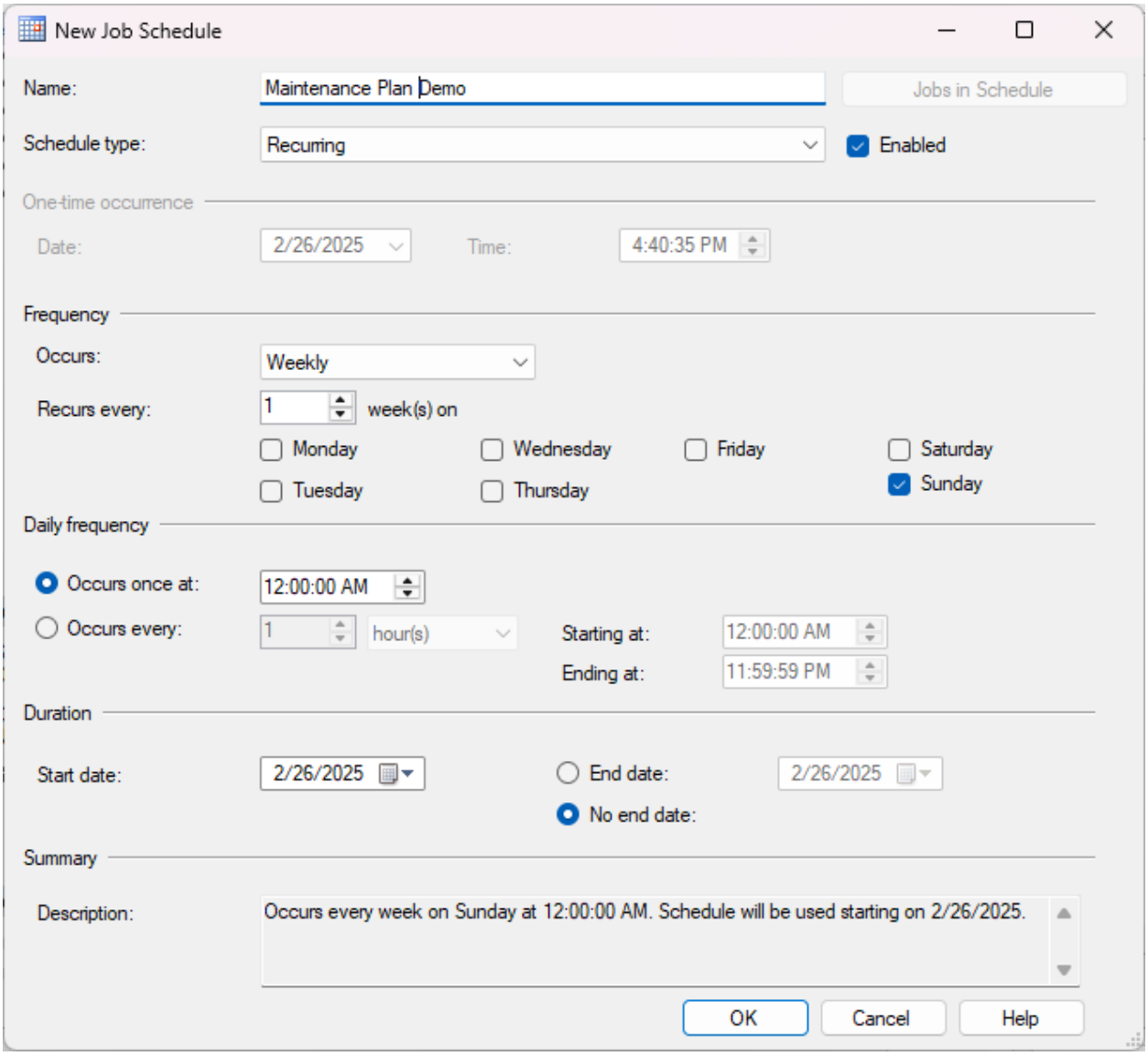
- The image shows the first screen of the Maintenance Plan Wizard in SQL Server Management Studio (SSMS).
- You need to specify a name for your maintenance plan and a run-as account.
- Most maintenance tasks will run as the SQL Server Agent service account, but for security purposes, some tasks may need to run as a different account.
- For example, if you need to back up to a file share accessible only by a specific account, you would use a proxy user, which is a component of the SQL Server Agent.

• What is a proxy account?

- A proxy account is an account with stored credentials that the SQL Server Agent can use to execute specific job steps as a designated user.
- The login information for this user is stored as a credential in the SQL Server instance.
- Proxy accounts are typically used when specific job steps require very granular security rights.
- Suppose you have a SQL Server Agent job that needs to back up a database to a network file share.
- If the SQL Server Agent service account does not have access to the file share, you can create a proxy account with the necessary permissions.
- This proxy account can then be used to run the backup step, ensuring it has the required access rights.

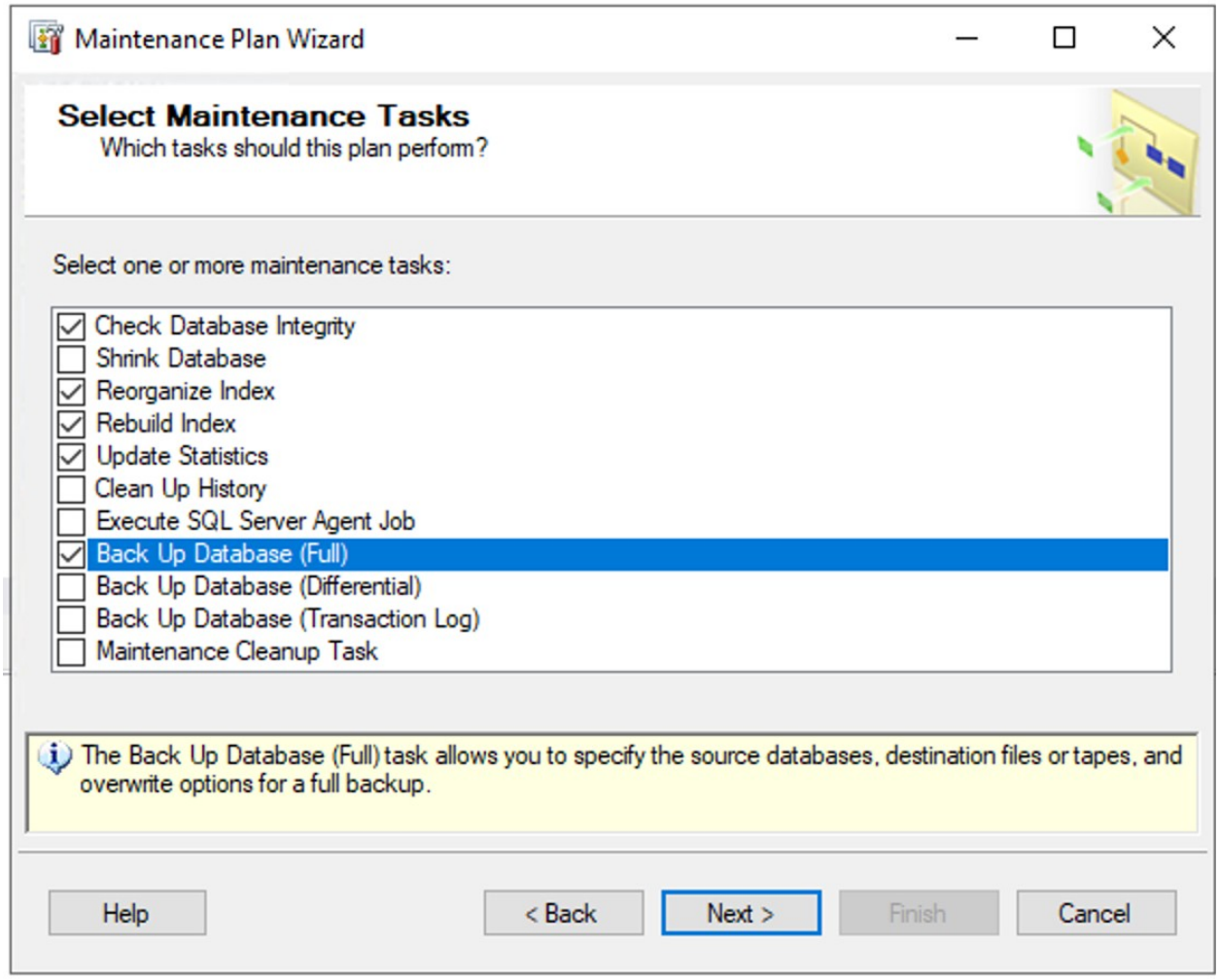
- Job schedules
- Job schedules are part of the job system in the msdb system database.
- SQL Server Agent jobs and schedules have a many-to-many relationship, meaning each job can have multiple schedules, and each schedule can be assigned to multiple jobs.
- However, the Maintenance Plan Wizard does not allow the creation of independent schedules.
- Instead, it creates a specific schedule for each maintenance plan.

  The following example shows the schedule for a weekly execution, but you also have the option to create a schedule with hourly or daily recurrence.



The next step is to select the maintenance tasks to add to the plan.

- The following example shows the operations available to be performed by your maintenance plan.



- Check database integrity - This task runs the DBCC CHECKDB command to validate the logical and physical consistency of each database page.
- You should perform this task regularly and align it with your backup retention window.
- Ensure you complete a consistency check before discarding any prior backups to prevent carrying over corruption.
- Shrink database - This task reduces the size of a database or transaction log file by moving data into free space on pages.
- Once enough space is freed, it can be returned to the file system.
- It is recommended not to include this action in regular maintenance as it causes severe index fragmentation, harming database performance.
- The operation is also very I/O and CPU intensive, which can significantly impact system performance.
- Reorganize/Rebuild index - This task checks the level of fragmentation in a database's indexes and either rebuilds or reorganizes the index based on the user-defined fragmentation level.
- Rebuilding an index also updates its statistics.

Update statistics - This task updates the column and index statistics used by SQL Server to build query execution plans.

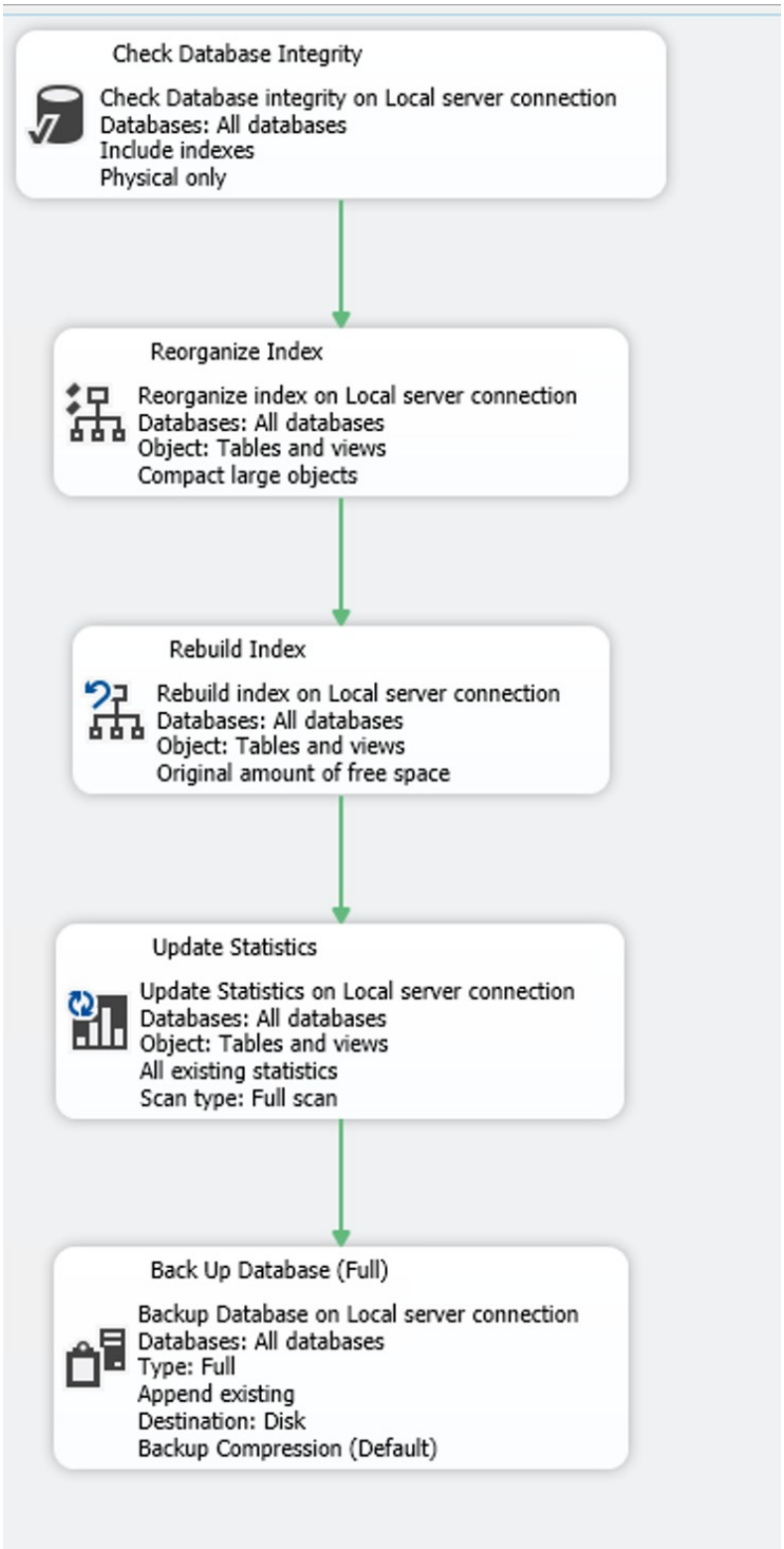
- Accurate statistics are crucial for the query optimizer to make the best decisions.
- You can choose which tables and indexes to scan and the percentage or number of rows to scan.
- The default sampling rate is usually sufficient, but you may need more detailed statistics for specific tables.
- Cleanup history - This task deletes the history of backup and restore operations from the msdb database, as well as the history of SQL Server Agent jobs.
- It helps manage the size of the msdb database.

Execute SQL Server Agent job - This task runs a user-defined SQL Server Agent job.

Backup Database (Full/Differential/Log) - This task backs up databases on a SQL Server instance.

- A full backup captures the entire database and serves as the starting point for a restore.
- Differential backups capture the pages that have changed since the last full backup, providing an incremental restore point.

- Transaction log backups capture the active pages in your transaction log, allowing you to define your recovery point objective.
- Note that transaction log backups cannot be performed on databases in SIMPLE recovery mode.
- For example, If you take a full backup on Sunday and a differential backup each weeknight, to restore your database to noon on Thursday, you would restore Sunday's full backup, Wednesday's differential backup, and the transaction log backups from Wednesday's differential to Thursday at noon.
- Maintenance Cleanup Tasks - This task removes old files related to maintenance plans, including text reports and backup files.
- It only removes backups in the specified folders, so any subfolders must be explicitly listed or they will be skipped.
- Each task can be scoped to user databases, system databases, or a custom selection of databases, and each has specific configuration options.



- Upon creation, the plan will appear as a job in the SQL Server Agent.

  If you added a schedule either during the creation process or after, that job will be executed and the maintenance tasks will be performed.

Multiserver environment

- In a multiserver environment, SQL Server Agent allows you to designate one server as a central server (known as MSX) that can execute jobs on other servers, known as target servers (TSX).
- The central server stores the main source of the jobs and distributes them to the target servers.
- Target servers periodically connect to the central server to update their job schedules.
- This setup enables you to define a job once and deploy it across your enterprise.

- For example, you can configure database maintenance tasks on the primary server and push them out to a group of target servers, ensuring consistent deployment.

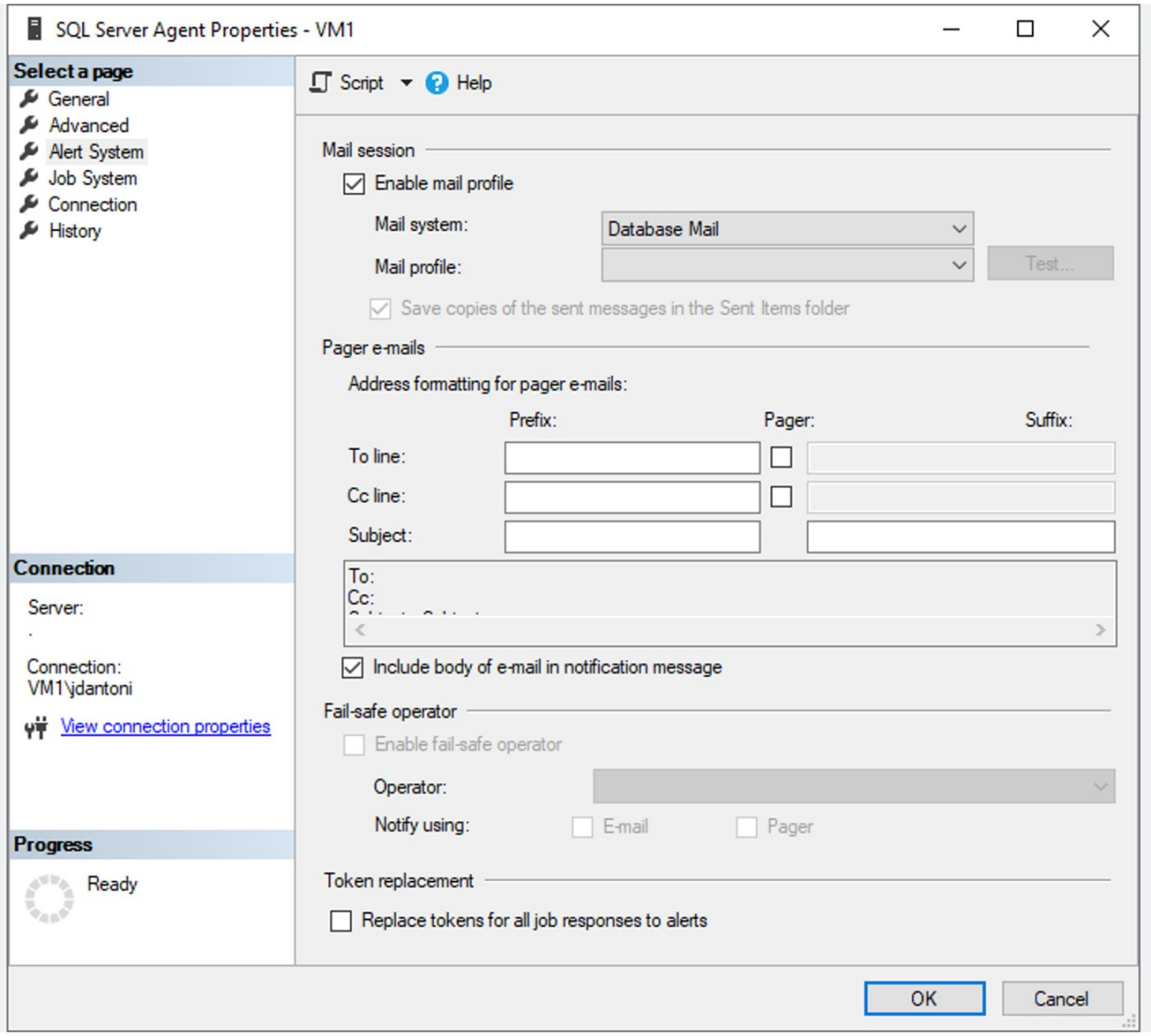
Describe task status notifications

- A crucial part of automation is providing notifications for job failures or specific system errors.
- SQL Server Agent facilitates this through a set of objects, with alerting commonly done via email using SQL Server's Database Mail functionality.
- The key objects in this workflow are:
- Operators: Aliases for individuals or groups who receive notifications.
- Notifications: Inform an operator about the completion, success, or failure of a job.
- Alerts: Assigned to an operator for either a notification or a defined error condition.

Operators

Operators act as aliases for users or groups configured to receive notifications of job completions or alerts from the error log.

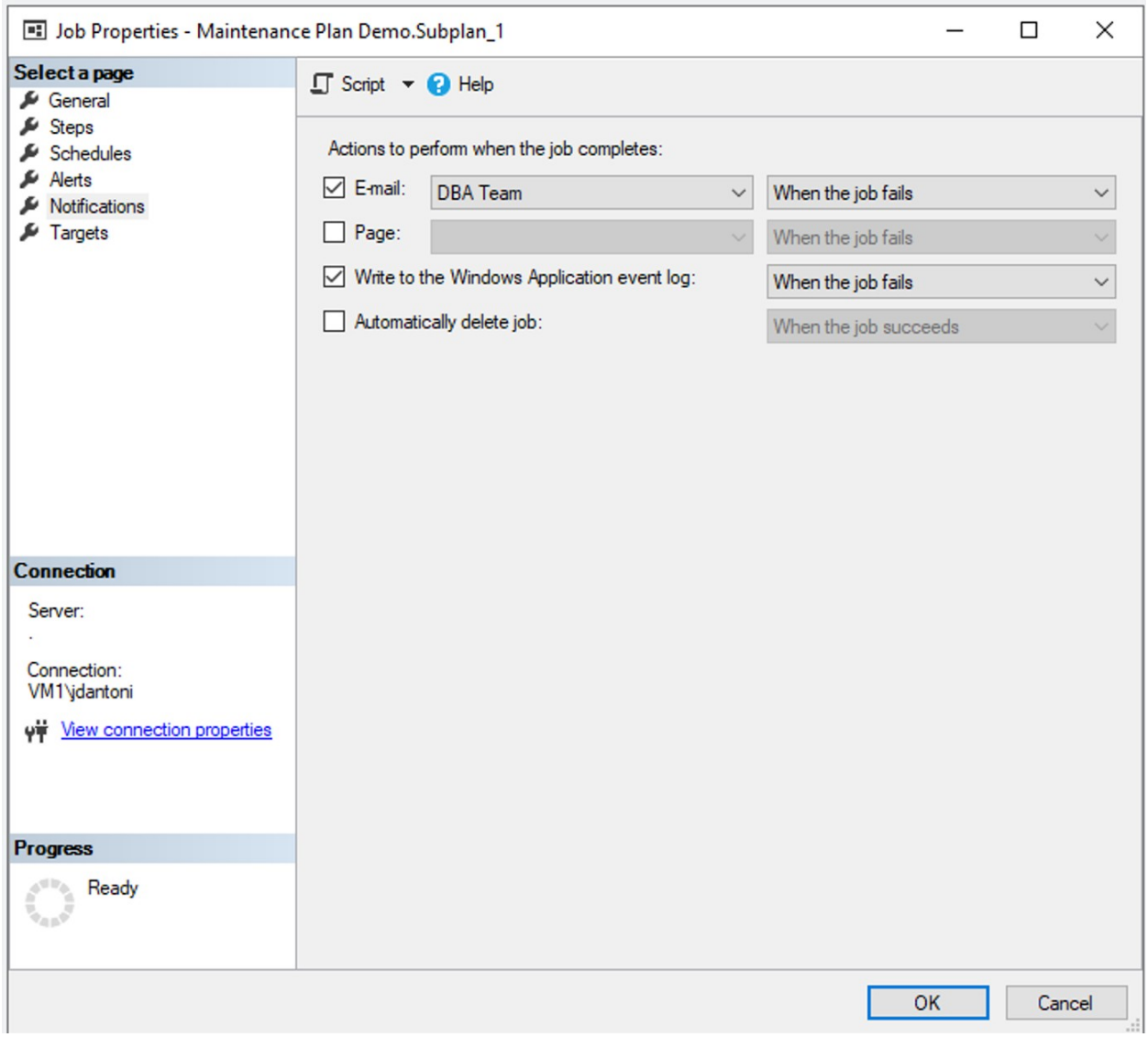
- An operator is defined by a name and contact information, typically mapping to an email group.
- Using email groups provides redundancy, ensuring notifications aren't missed if someone is unavailable.
- It also simplifies updates when employees leave the organization.
- To send emails to an operator, you need to enable the SQL Server Agent's email profile, as shown below:



Notifications

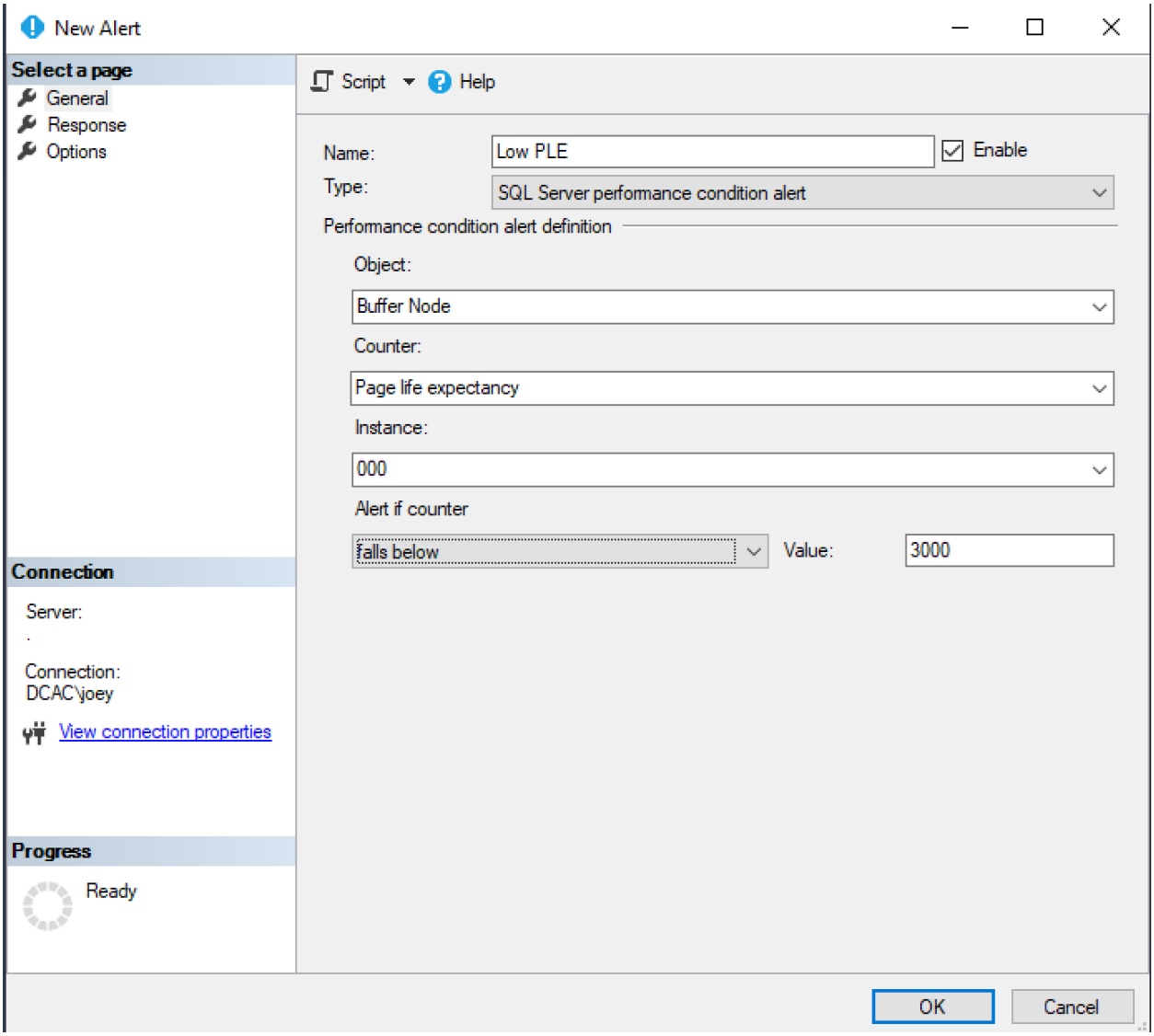
- Notifications are part of each SQL Server Agent job.
- You can choose to send a notification on job completion, failure, or success.

- Most DBAs notify only on failure to avoid an influx of notifications for successful jobs.
- Notifications depend on an existing operator to send the alert.



Alerts

- SQL Server Agent alerts enable Proactive Monitoring of your SQL Server.
- The agent reads the Windows application log and notifies an operator when it finds an error number for which an alert is defined.
- Besides monitoring the Windows application log, you can set up alerts for SQL Server performance conditions and Windows Management Instrumentation (WMI) events.
- You can specify alerts for one or more events.
- A common practice is to raise alerts for all SQL Server errors of level 16 and higher and add alerts for specific critical storage errors or Availability Group failovers.
- Another example is to alert on performance conditions like high CPU utilization or low Page Life Expectancy.
- DBAs may also want to be notified of certain server conditions, such as CPU utilization over 90% for five minutes or low Page Life Expectancy.
- This is done by creating performance condition alerts based on Windows Performance Monitor (perfmon) metrics tracked within the SQL Server database engine.
- You can access the alert configuration screen by right-clicking SQL Server Agent (if it is running) and choosing New | Alert.



- You have options for responding to performance conditions: notify an operator via email, which is the most common approach, or execute another SQL Server Agent job to resolve the issue.

  Executing another job is useful for well-known conditions that can be handled without manual intervention. For example, you could create an alert for SQL Server storage error conditions (errors 823, 824, 825) and execute a job to perform a database consistency check.
- Notifications for these alerts use the same SQL Server Agent subsystem.

Knowledge check

- Choose the best response for each of the questions below.
- Check your knowledge
- 1.

• What has to be configured before the SQL Server Agent can send email?

A mail profile

An agent job

An alert

- 2.

• **Which system database stores SQL Server Agent jobs and their information?**

Msdb

Master

Model

• 3.

• **Which operation recalculates the statistics on an index?**

Rebuild

Reorganize

Shrinking a file group

• 4.

• **What built-in option removes old files related to maintenance plans?**

Cleanup history task

Maintenance cleanup task

Execute SQL Server agent job task

Exercise: Create a CPU status alert for a SQL Server

In this exercise, you'll learn how to create a CPU status alert for a SQL Server on Azure.

Note

- To complete this exercise, you'll need an Azure subscription.
- Launch the exercise and follow the instructions.

Launch Exercise

Summary

- **1 minute**
- The SQL Server Agent provides a robust mechanism for managing and maintaining your SQL Server and Azure SQL managed instance deployments.
- In addition to providing job scheduling, it provides alerting and some monitoring for your databases.
- Maintenance plans can be used to provide backups, index and statistics maintenance, and log management activity.
- Note that the SQL Server Agent is only available on SQL Server and Azure SQL Managed Instance, but not Azure SQL Database.
- Now that you've reviewed this module, you should be able to:
- Describe what maintenance activities you should perform on your databases
- Describe how to configure notifications and alerts on SQL Server Agent jobs and SQL Server
- Describe how to configure notifications alerts based on performance monitor values

Manage Azure PaaS tasks using automation

Intermediate

Data Analyst

Data Engineer

Database Administrator

Azure

- **Azure SQL Database**
- **SQL Server**
- Explore automation for Azure SQL platform.

- Configure elastic jobs, explore Azure Automation, and evaluate different strategies for monitoring automation tasks.

Learning objectives

- After completing this module, you will be able to:
- **Undersand the benefits of Azure policy**
- Explore the capabilities of Azure Automation
- **Configure elastic jobs**

Use Logic Apps for database workflow

Prerequisites

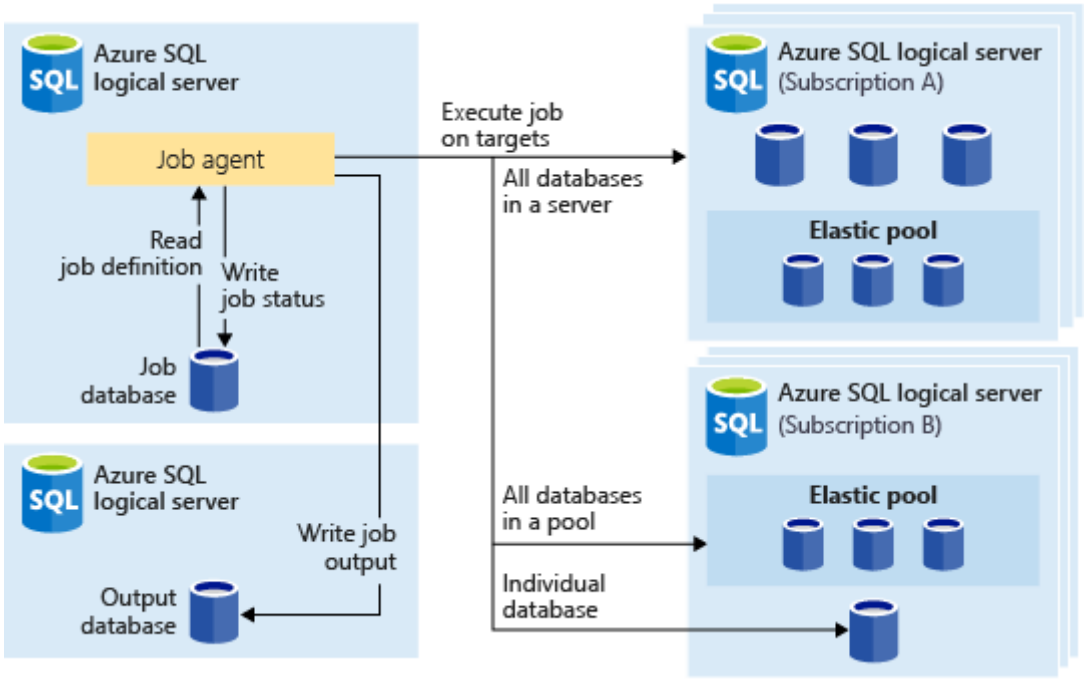
- Ability to use tools for running queries against a Microsoft SQL database, either on-premises on cloud-based.
- Ability to write code in the SQL language, particularly the Microsoft T-SQL dialect, at a basic level.
- Experience creating and configuring resources using the Azure portal.
- Familiarly with Open Source database systems on Azure.

Introduction

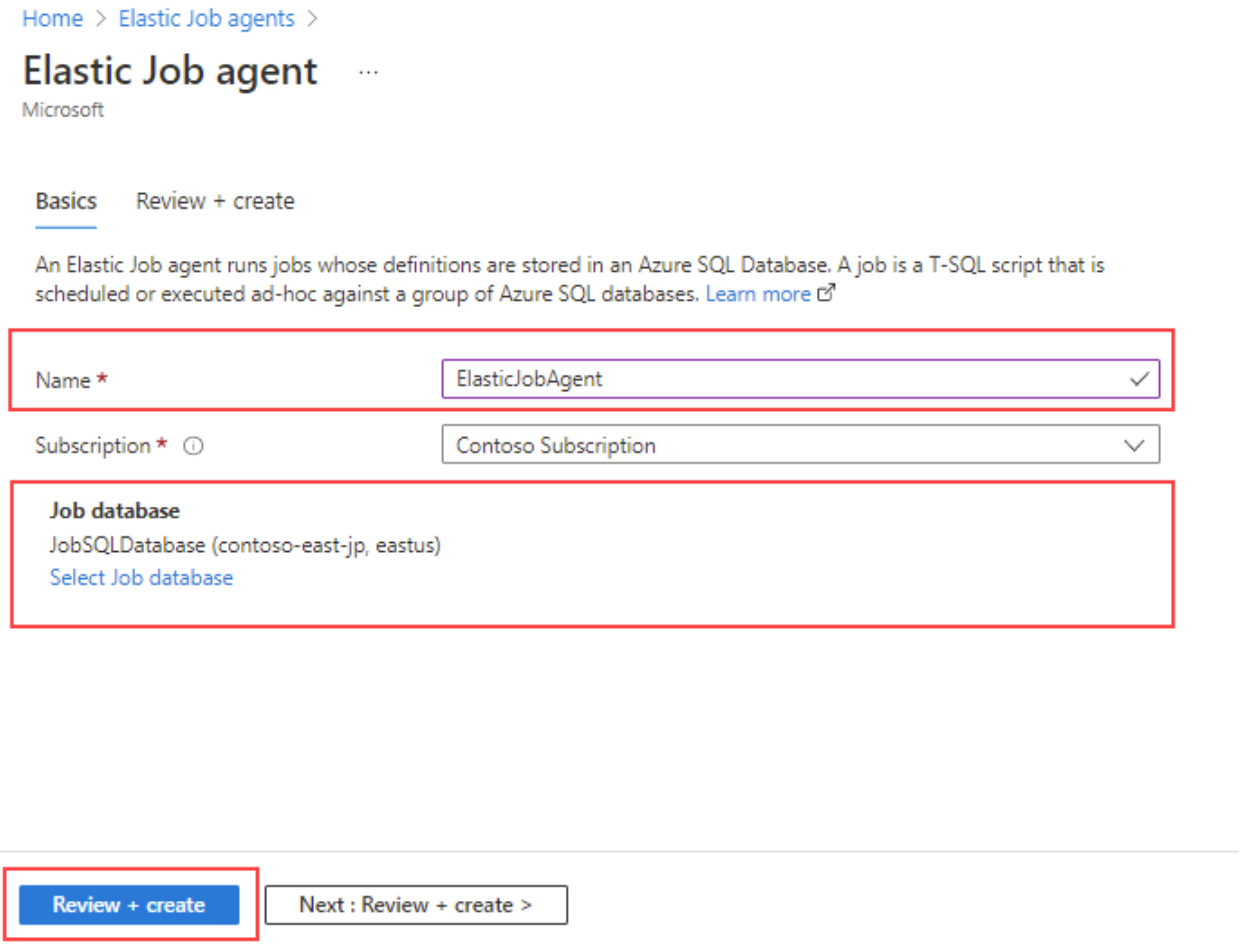
- **1 minute**
- If you're proficient in SQL Server, you're likely familiar with the capabilities of the SQL Server Agent.
- SQL Server Agent is also available on Azure SQL Managed Instance.
- However, if you're using Azure SQL Database, you'll need an alternative scheduling mechanism, as both the msdb database and the SQL Server Agent are unavailable.
- In this module you'll learn about Azure Automation, Logic Apps, and elastic jobs, three approaches for automating jobs in PaaS.
- You'll evaluate different strategies for monitoring your automation tasks.
- This includes setting up alerts and notifications to stay informed about job statuses and system errors, using performance metrics to track the effectiveness of your automation, and employing tools to troubleshoot and optimize your automated processes

Explore Elastic jobs

- Many DBA's became familiar with Azure Automation because Azure SQL Database initially lacked capabilities for scheduled jobs.
- The elastic jobs capability allows you to run a set of T-SQL scripts against a collection of servers or databases either as a one-time job or on a defined schedule.
- Elastic Jobs function similarly to SQL Server Agent jobs but are limited to executing T-SQL.
- They work across all tiers of Azure SQL Database.
- SQL Agent jobs continue to be used for task automation in SQL Server and are also included with Azure SQL Managed Instances.



- To configure Elastic Jobs, you need a Job Agent and a dedicated database to manage your jobs.
- The recommended service tier for the job database is S1 or higher, depending on the number and frequency of jobs you're executing.
- Let's review the components of Elastic Jobs:
- Elastic job agent: Your Azure resource for running and managing jobs.
- Job database: A dedicated database to manage your jobs.
- Target group: A collection of servers, elastic pools, and single databases where a job will be run.
- Job: One or more T-SQL scripts that make up a job step.
- If a server or elastic pool is the target, you need to create a credential within the master database of the server or pool so the job agent can enumerate the databases.
- For a single database, only a database credential is needed.
- Credentials should have the least privileges necessary to perform the job step.



You can create an elastic job agent through the Azure portal, PowerShell, or REST API.
To create an Elastic Job agent from the Azure portal, navigate to the Elastic Job Agent page.

- Here, provide a name for your agent and specify an SQL database for your job database.
- You can create a target group using PowerShell, T-SQL, or Azure CLI.
- The following snippet creates a MyServerGroup target group, including all databases on the server at the time of execution.
- This snippet assumes the variables $jobAgent and $targetServerName were previously provided.

PowerShell

```
# create MyServerGroup target group
$serverGroup = $jobAgent | New-AzSqlElasticJobTargetGroup -Name 'MyServerGroup'
$serverGroup | Add-AzSqlElasticJobTarget -ServerName $targetServerName -RefreshCredentialName $masterCred.
CredentialName
```

- This script sets up an elastic job target group and adds a target server to it, allowing you to run jobs against the specified server.
- The following code snippet creates an elastic job and adds job steps using PowerShell.
- Step1 is responsible for creating the MyTable table if it doesn't already exist.

PowerShell

Write-Output "Creating a new job..."

```
$jobName = "MyFirstElasticJob"
$job = $jobAgent | New-AzSqlElasticJob -Name $jobName -RunOnce
```

Write-Output "Creating job steps for $($jobName) job..."

```
$sqlText1 = "IF NOT EXISTS (SELECT * FROM sys.tables WHERE object_id = object_id('MyTable')) CREATE TABLE [dbo].
[MyTable]([Id] [int] NOT NULL);"
```

```
$job | Add-AzSqlElasticJobStep -Name "Step1" -TargetGroupName $serverGroup. TargetGroupName -CredentialName
$jobCred. CredentialName -CommandText $sqlText1
```

- The T-SQL scripts executed by elastic jobs should be idempotent.
- This means that if the job runs multiple times, whether accidentally or due to a job failure, it won't fail or produce unintended results.
- You should be able to run the same script multiple times without encountering errors.
- Finally, run the elastic job MyFirstElasticJob using PowerShell.

PowerShell

Write-Output "Start the job..."

```
$jobExecution = $job | Start-AzSqlElasticJob
$jobExecution
Use case scenarios
```

- Elastic jobs can be used in the following scenarios:
- Automate management tasks to run on a specific schedule.
- Deploy schema changes seamlessly.
- Move data efficiently.
- Collect and aggregate data for reporting or other purposes.
- Load data from Azure Blob Storage.
- Configure jobs to execute across multiple databases on a recurring basis, such as during off-peak hours.
- Process data across numerous databases, such as telemetry collection, and compile results into a single destination table for further analysis.
- SQL Agent jobs and Elastic Jobs serve different purposes across Azure SQL platforms.
- SQL Agent jobs, available in SQL Server and Azure SQL Managed Instances, offer robust scheduling and automation.
- Elastic Jobs, designed for Azure SQL Database and elastic pools, run T-SQL scripts across multiple databases.

Understand Azure Automation

- Azure offers several ways to automate processes.
- Azure Functions and Logic Apps are services that enable serverless workloads, creating workflows composed of steps to execute complex tasks.
- For example, a Logic App can populate a table in an Azure SQL Database when an entry is made in a SharePoint list.
- A full explanation of these services is beyond the scope of this course.
- For more control and granularity in your automation, Azure Automation provides process automation, configuration management, full integration with Azure platform options (such as role-based access control and Microsoft Entra ID), and the ability to manage both Azure and on-premises resources.
- One unique benefit of Azure Automation is its capability to manage resources within Azure or on-premises VMs.
- For instance, if you have a VM that is kept in a down state for cost savings, you can use Azure Automation with hybrid runbooks to start the VM, perform a SQL Server backup, and then shut down the VM.
- Another common use of Azure Automation is for periodic maintenance operations, such as purging stale or old data or reindexing an SQL database.
- Azure Automation components
- Azure Automation supports both automation and configuration management activities.
- While we'll focus on the automation components, Azure Automation can also manage server updates and Desired State Configuration.
- Here are the key components you need to execute automated tasks.
- Runbooks: Runbooks are the units of execution in Azure Automation.
- They can be defined as graphical runbooks based on PowerShell, PowerShell scripts, or Python scripts.
- PowerShell runbooks are most commonly used to manage Azure SQL resources.
- Modules: Azure Automation provides an execution context for the PowerShell or Python code in your runbook.
- To execute your code, you need to import the necessary modules.
- For example, to run the Get-AzSqlDatabase PowerShell cmdlet, you would import the Az.SQL PowerShell module into your automation account.
- Credentials: Credentials store sensitive information that runbooks or configurations can use at runtime.
- Schedules: Schedules are linked to runbooks and trigger a runbook at a specific time.

- Azure Policy
- Group Policies (GPOs) have long been used by Windows Server administrators to manage security and ensure consistency across Windows Server environments.
- Examples include enforcing password complexity and mapping shared network drives.
- Azure Policy offers similar governance for Azure resources, enforcing rules like region limitations, naming standards, and resource sizes.
- Policies can be applied at various levels, such as management groups, subscriptions, or resource groups.
- They can also be grouped into initiatives for broader application.
- Another use of Azure Policy is resource tagging, which stores metadata in key-value pairs.
- For example, a policy might require all resources to have tags for environment and cost center, blocking deployment if tags are missing.
- Azure subscriptions and tags
- Organizations use multiple subscriptions for various reasons, such as budget management, security, or resource isolation.
- For instance, an organization might separate internal and customer-facing resources into different subscriptions to simplify billing and isolate internal resources.
- These subscriptions can be managed together in a management group, allowing for unified policy and compliance management across subscriptions.
- Tags are metadata used to describe your Azure resources better.
- Stored as key: value pairs, tags appear in the Azure portal alongside your resources.
- Because they're associated with the resource, you can filter PowerShell or Azure CLI commands based on tags, similar to using a WHERE clause in a SQL query.
- Here's a basic example:

PowerShell

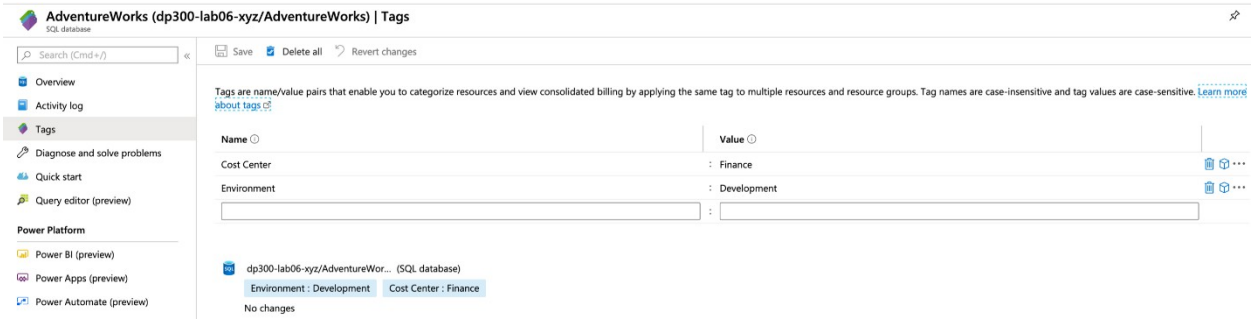
```
$rg=(get-AzResourceGroup)
```

```
$rg=($rg|where-object {($_.tags['Use'] -ne 'Internal')}).ResourceGroupName
```

In the second line of this code sample, the list of resource groups is filtered by the tag called 'Use', returning only those resource groups where the tag value isn't 'Internal'.

- Tags can be applied in the Azure portal, programmatically via PowerShell, Azure CLI, or as part of your deployment process.
- They can be applied at the subscription, resource group, or individual resource level and can be modified at any time.
- Azure supports up to 50 tags per resource.
- Tags are also included in Azure billing information, making it easier for management to break down charges by cost center.
- Tags are found in the overview section of the blade for every Azure resource.

To add tags to a resource using the Azure portal, select Tags, enter the key and value for your tag, and then select Save.



- The following example shows how to add tags using PowerShell.

PowerShell

```
$tags = @{"Dept"="Finance"; "Status"="Normal"}
```

```
$resource = Get-AzResource -Name demoStorage -ResourceGroup demoGroup
```

```
New-AzTag -ResourceId $resource.id -Tag $tags
```

- The following example shows how to add tags using Azure CLI.

Azure CLI

```
az resource tag --tags 'Dept=IT' 'Environment=Test' -g examplegroup -n examplevnet `

--resource-type "Microsoft. Network/virtualNetworks"
```

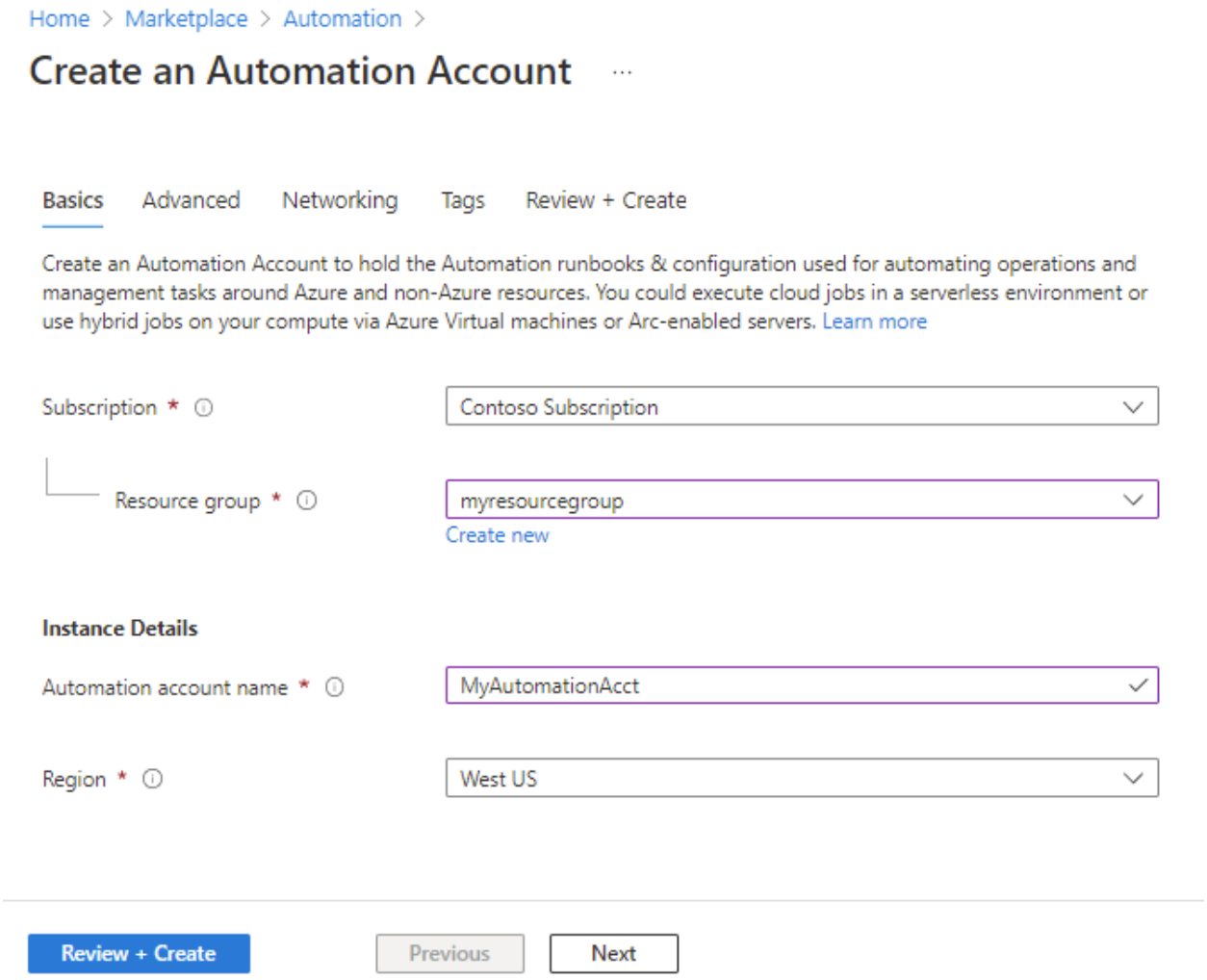
- Tags enable customers to organize Azure resources and management hierarchy into a taxonomy.
- Note
- Tags are stored as plain text, so never add sensitive values to them.
- Sensitive information could be exposed through various methods, including cost reports, tag taxonomies, deployment histories, exported templates, and monitoring logs.

Build an automation runbook

Let's look at the steps required to create an automation account and an automation runbook.

Create an automation account

To create an automation runbook, you first need to create an automation account.

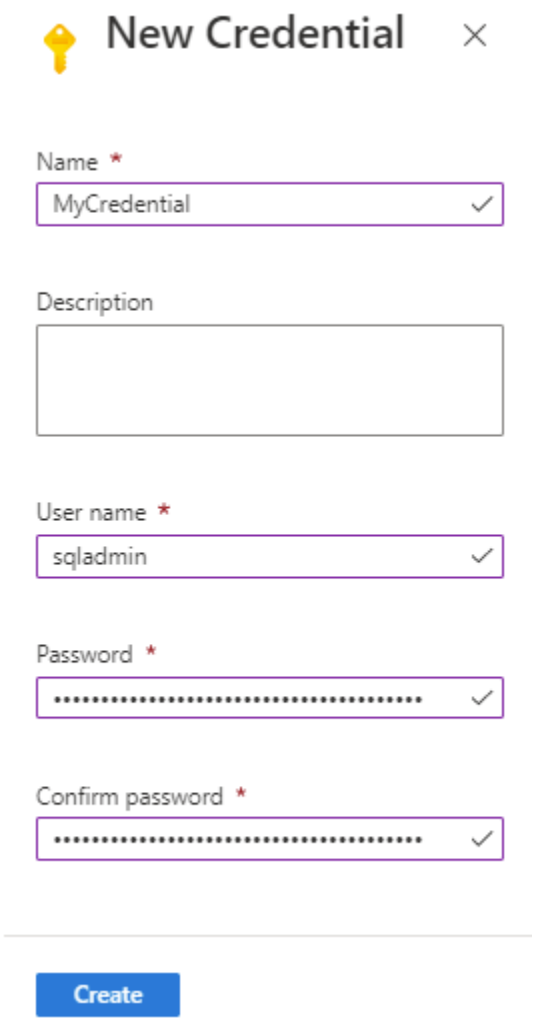


- In this example, you connect to an Azure SQL Database using PowerShell.
- This requires importing modules to support the necessary cmdlets.
- Before creating your runbook, import the required modules into your Azure Automation account.
- Navigate to the Shared Resources section of the main blade for your automation account and select Modules.
- Import the Az. Accounts module, as the Az.SQL module depends on it.

- Search for the module in the gallery as shown above.
- After selecting the module, select Import, as shown below.
- This imports the module into your account.
- Repeat this process for the Az.SQL module.

Next, you can optionally create a credential for your runbook to use.
Create a credential by selecting on Credentials in the Shared Resources section of the main blade of your automation account.

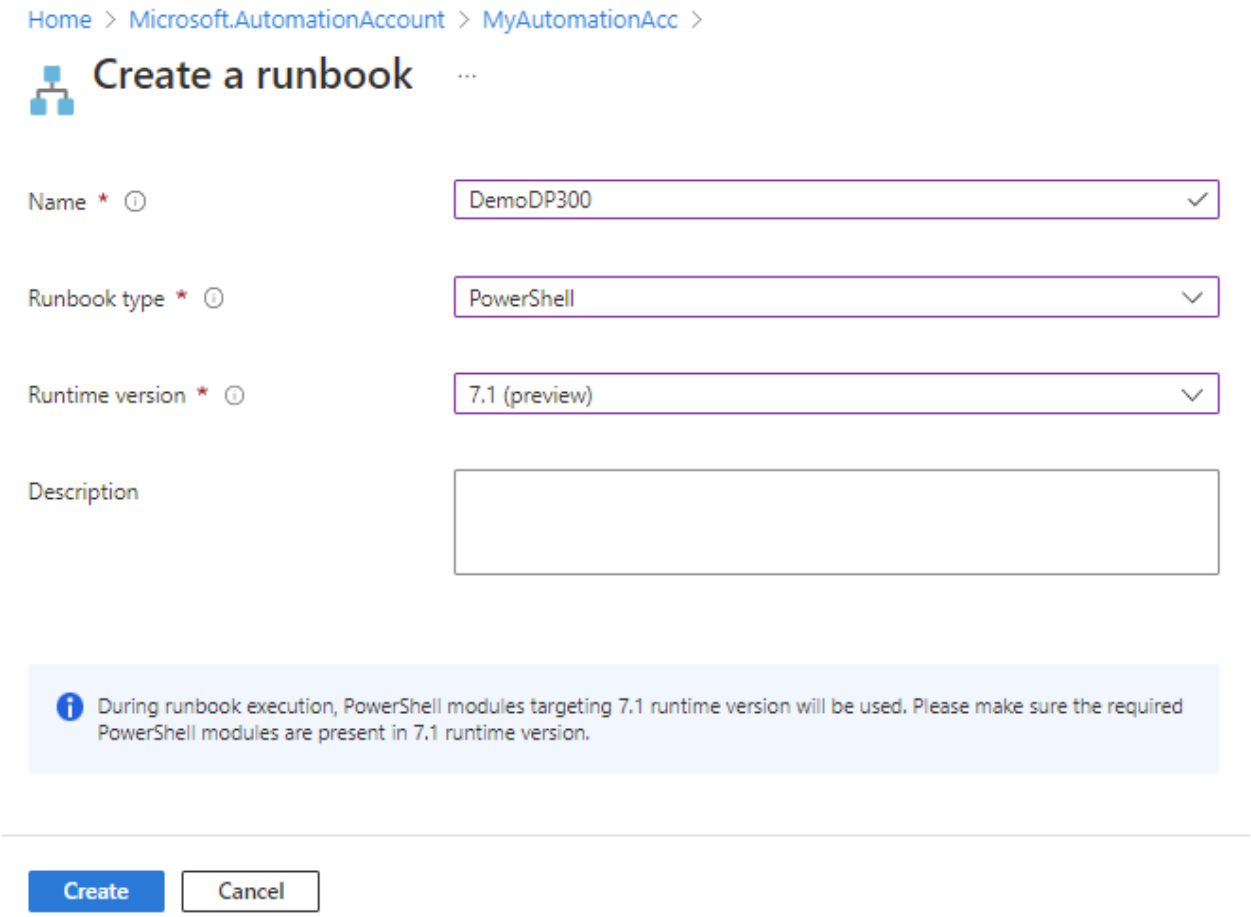
- Creating a credential isn't mandatory for using Azure Automation, but this example includes one.



Create a runbook

- Automation executes your runbooks based on the logic defined inside them.
- If a runbook is interrupted, it restarts at the beginning.
- This behavior requires you to write runbooks that support being restarted if transient issues occur.

On the Process Automation section of your Automation Account, select Runbooks, to create a runbook.



To create a runbook, you need to provide a name, the type of runbook, the runtime version, and optionally a description.

- Since this example specifies PowerShell as the type, a PowerShell editor opens.
- On the editor, provide the following PowerShell code.
- We use system-managed identity to log in to Azure.
- You need to enable appropriate RBAC permissions for the system identity of this Automation account.
- Otherwise, the runbook might fail.

```
PowerShell
# Uses the system-managed identity to log in to Azure
try {
Write-Output "Logging in to Azure..."
Connect-AzAccount -Identity
} catch {
Write-Error -Message $_.Exception
throw $_.Exception
}

# Get SQL Database name
$dbname = (Get-AzSQLDatabase -ResourceGroupName 'SQLDB' -ServerName 'GSData' -DatabaseName
'GSData').DatabaseName

# Set SQL Server name
$AzureSQLServerName = $dbname + ".database.windows.net"
```

```
# Get SQL credentials
$Cred = Get-AutomationPSCredential -Name "SQLUser"
```

```
# Execute SQL query
$SQLOutput = $(Invoke-Sqlcmd -ServerInstance $AzureSQLServerName -Username $Cred. UserName -Password $Cred.
GetNetworkCredential().Password -Database $dbname -Query "SELECT * FROM INFORMATION_SCHEMA.TABLES"
-Verbose) 4>&1
```

Write-Output $SQLOutput

- In this example, we use system-managed identity to log in to Azure, retrieves information about an Azure SQL Database, runs a query, and returns the results.
- You can select Test pane in the code editor in the Azure portal.
- This allows you to test your code in the context of Azure Automation.
- A typical development process involves creating your PowerShell code locally and then testing it within the automation environment.
- This helps separate any PowerShell errors from those generated by the automation context.
- Always test your code within automation to ensure there are no errors in the code itself.

- Automate database workflows by using Logic Apps
- Azure Logic Apps is a cloud-based platform designed to create and run automated workflows that integrate your apps, data, services, and systems.
- This platform enables you to quickly develop highly scalable integration solutions for both enterprise and business-to-business (B2B) scenarios.
- As part of Azure Integration Services, Azure Logic Apps simplifies connecting legacy, modern, and cutting-edges systems across cloud, on-premises, and hybrid environments.
- Here are some example tasks, business processes, and workloads you can automate using Azure Logic Apps:
- Schedule and send email notifications using Office 365 when specific events occur, such as a new file upload.
- Route and process customer orders across on-premises systems and cloud services.

Move uploaded files from an SFTP or FTP server to Azure Storage.
Monitor tweets, analyze sentiment, and create alerts or tasks for items that need review.

• Why use Azure Logic Apps?

- Azure Logic Apps provides prebuilt, Microsoft-managed API connectors and built-in operations, making it easier and quicker to connect and integrate apps, data, services, and systems.
- This allows you to focus on designing and implementing your solution's business logic and functionality, rather than figuring out how to access your resources.
- Typically, you won't need to write any code.
- However, if you do, you can create code snippets using Azure Functions and run them from your workflow.
- You can also use the Inline Code action to run code snippets directly within your workflow.

If your workflow needs to interact with events from Azure services, custom apps, or other solutions, you can monitor, route, and publish events using Azure Event Grid.
Logic Apps is fully managed by Microsoft Azure, freeing you from concerns about hosting, scaling, managing, monitoring, and maintaining solutions built with these services.
By using these capabilities to create "serverless" apps and solutions, you can focus solely on the business logic and functionality.

- These services automatically scale to meet your needs, speed up integrations, and help you build robust cloud apps with little to no code.

SQL Server connector

The SQL Server connector in Azure Logic Apps allows you to access your SQL database and create automated workflows triggered by events in your SQL database or other systems.

- This enables you to manage your SQL data and resources efficiently.
- For example, you can use actions to get, insert, and delete data, as well as run SQL queries and stored procedures.
- You can create a workflow that checks for new records in a non-SQL database, processes the data, creates new records in your SQL database, and sends email alerts about the new records.
- The SQL Server connector supports the following SQL editions:

- **SQL Server**
- **Azure SQL Database**
- **Azure SQL Managed Instance**
- The SQL Server connector requires that your tables contain data so that SQL connector operations can return results when called.
- For instance, if you use Azure SQL Database, you can use the included sample databases to try out the SQL connector operations.
- For an SQL database in Azure, the connection string has the following format:

  Server=tcp:{server-name}.database.windows.net,1433; Initial Catalog={database-name}; Persist Security Info=False; User ID={user-name}; Password={password}; MultipleActiveResultSets=False; Encrypt=True; TrustServerCertificate=False; Connection Timeout=30;

  Alternatively, you can check the connection string for your Azure SQL Database in the Azure portal.

  In the Overview section for your database, select Show database connection strings under the Connection strings property.

  If you want to start your workflow with a SQL Server trigger operation, you need to begin with a blank workflow.
- The SQL Server connector is available for logic app workflows in multitenant Azure Logic Apps and single-tenant Azure Logic Apps:
- Consumption workflows in multi-tenant Azure Logic Apps: This connector is available only as a managed connector.
- For more information, review the managed SQL Server connector operations page.
- Standard workflows in single-tenant Azure Logic Apps: This connector is available as both a managed connector and a built-in connector designed to run in the same process as the single-tenant Azure Logic Apps runtime.
- The built-in connector supports triggers (for example, when a row is inserted, modified, updated, or deleted) and multiple actions such as Execute query, Execute stored procedure, Get rows, Insert row, Update rows, and Delete rows.

  Create a logic app workflow

  The following steps use the Azure portal to create logic app workflow.

Add a SQL Server trigger

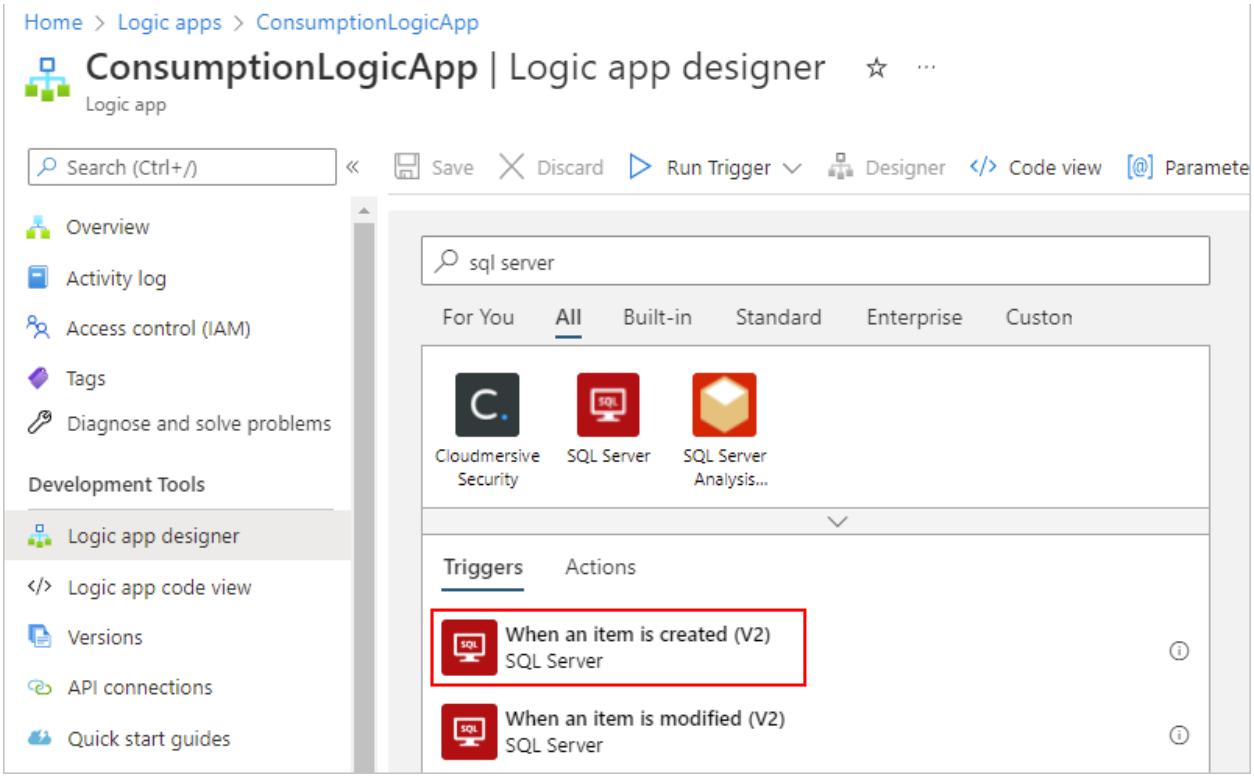
The following steps use the Azure portal, but with the appropriate Azure Logic Apps extension, you can also use Visual Studio Code to create logic app workflows:

- In the Azure portal, open your blank logic app workflow in the designer.

  Find and select the managed SQL Server connector trigger that you want to use.

  Under the designer search box, select All.
- In the designer search box, enter sql server.

  From the triggers list, select the SQL trigger that you want.
- This example uses the trigger named When an item is created.



If you're connecting to your SQL database for the first time, you're prompted to create your SQL database connection now. After you create this connection, you can continue with the next step.

- In the trigger properties, specify the interval and frequency for how often the trigger checks the table.

To add other properties available for this trigger, open the Add new parameter list and select those properties.

Note

This trigger returns only one row from the selected table, and nothing else.

- To perform other tasks, continue by adding either a SQL Server connector action or >another action that performs the next task that you want in your logic app workflow.
- For example, to view the data in this row, you can add other actions that create a file that includes the fields from the returned row, and then send email alerts.
- To learn about other available actions for this connector, see the connector's reference page.

On the designer toolbar, select Save. this step automatically enables and publishes your logic app live in Azure

Add a SQL Server action

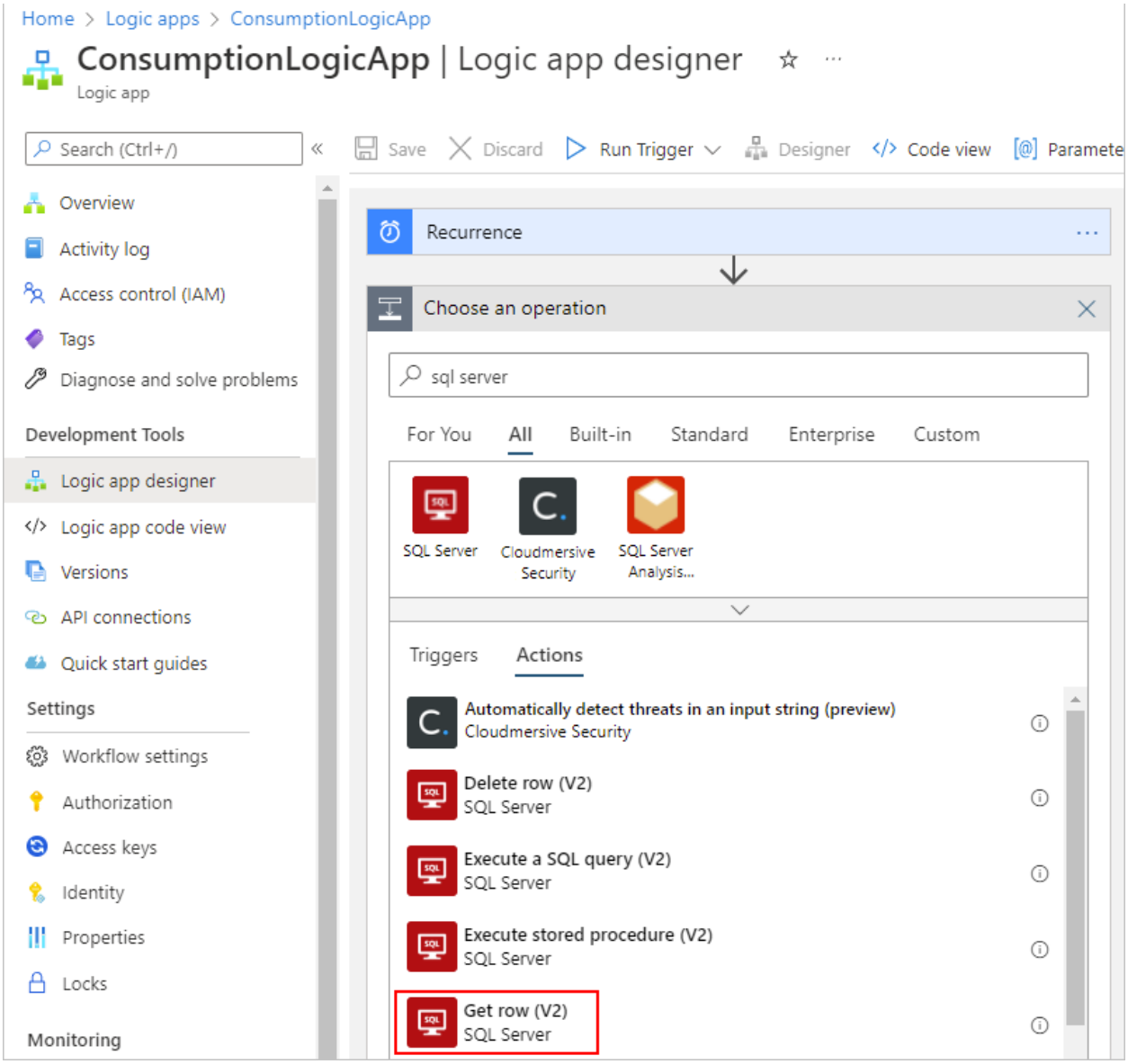
- The following steps use the Azure portal.
- In this example, the logic app workflow starts with the Recurrence trigger, and calls an action that gets a row from an SQL database.
- In the Azure portal, open your logic app workflow in the designer.
- Find and select the managed SQL Server connector action that you want to use.
- This example uses the action named Get row.

Under the trigger or action where you want to add the SQL action, select New step.
In the Choose an operation box, under the designer search box, select All.

- In the designer search box, enter sql server.

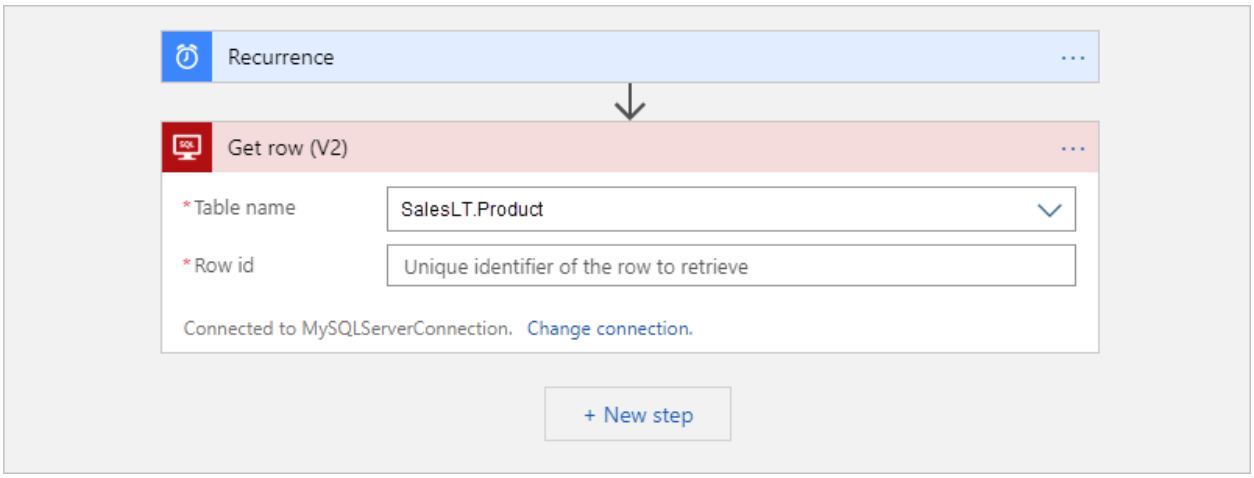
From the actions list, select the SQL Server action that you want.

- This example uses the Get row action, which gets a single record.



If you haven't already provided the SQL Server name and database name, provide those values. Otherwise, from the Table name list, select the table that you want to use.

- In the Row ID property, enter the ID for the record that you want.
- In this example, the table name is SalesLT.Product.



Note

This action returns only one row from the selected table, and nothing else.
When you're done, on the designer toolbar, select Save.

Connect to Azure SQL Database

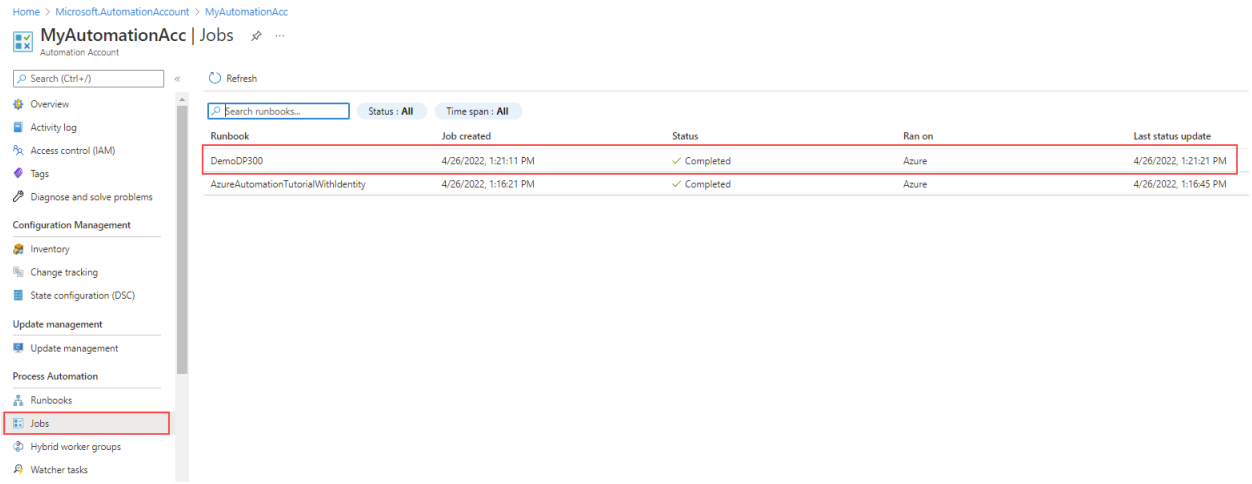
In the workflow designer, you must create a connection the first time you add a trigger or action for the first time.

- This information varies depending on the connection, for example:
- The name that you want to use for the new connection
- **The name for the system or server**
- **Your user or account credentials**
- **The authentication type to use**

Monitor automated tasks

- After automating your tasks, it's crucial to monitor them to ensure they're functioning correctly.
- This helps maximize the performance and availability of your services and proactively identify problems.
- Monitor Runbooks
- Your runbooks should be modular, with reusable, and easily restartable logic.
- Monitoring a runbook's progress ensures that its logic is executed correctly if issues arise.
- You can use a database, storage account, or shared file to monitor a runbook's progress.
- Ensure your runbook checks the last action performed before initiating the next action.
- Based on the results, the logic can either skip or continue specific tasks in the runbook.
- You can monitor runbook execution in the Azure portal.

Select Job in the Process Automation section of the main blade of your automation account.

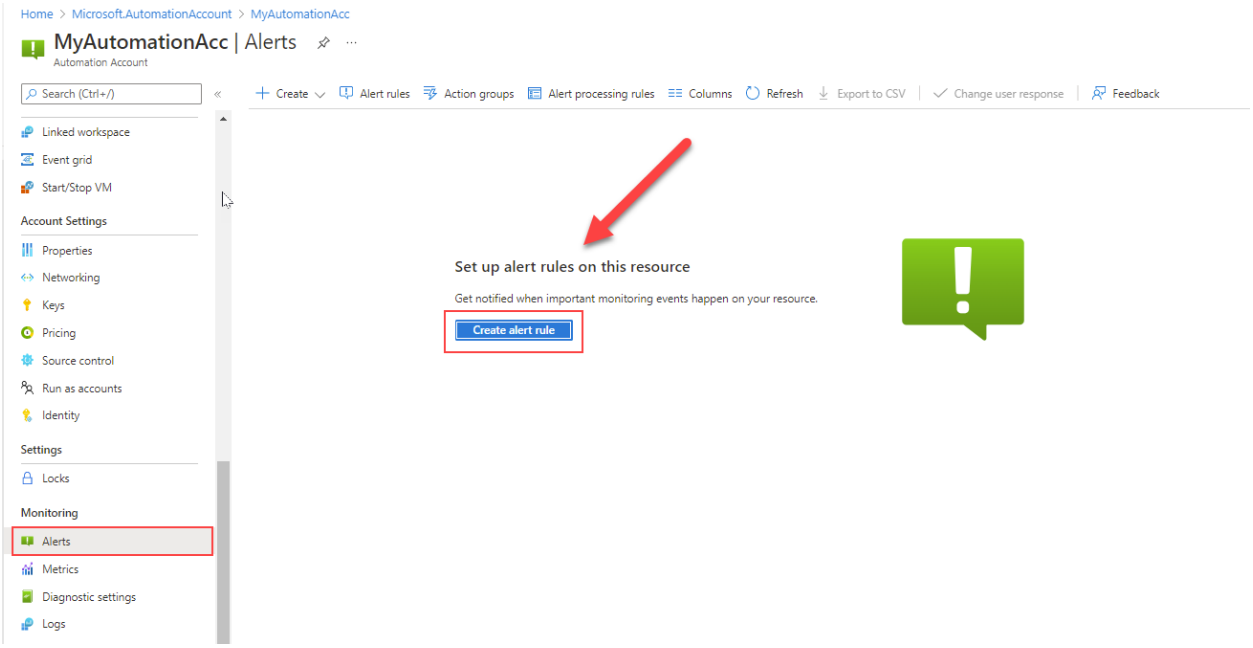


- As shown in the following image, the highlighted runbook was completed.
- To investigate the details and status, select the job.
- To learn more about how to troubleshoot runbooks problems, see Troubleshoot runbook issues.

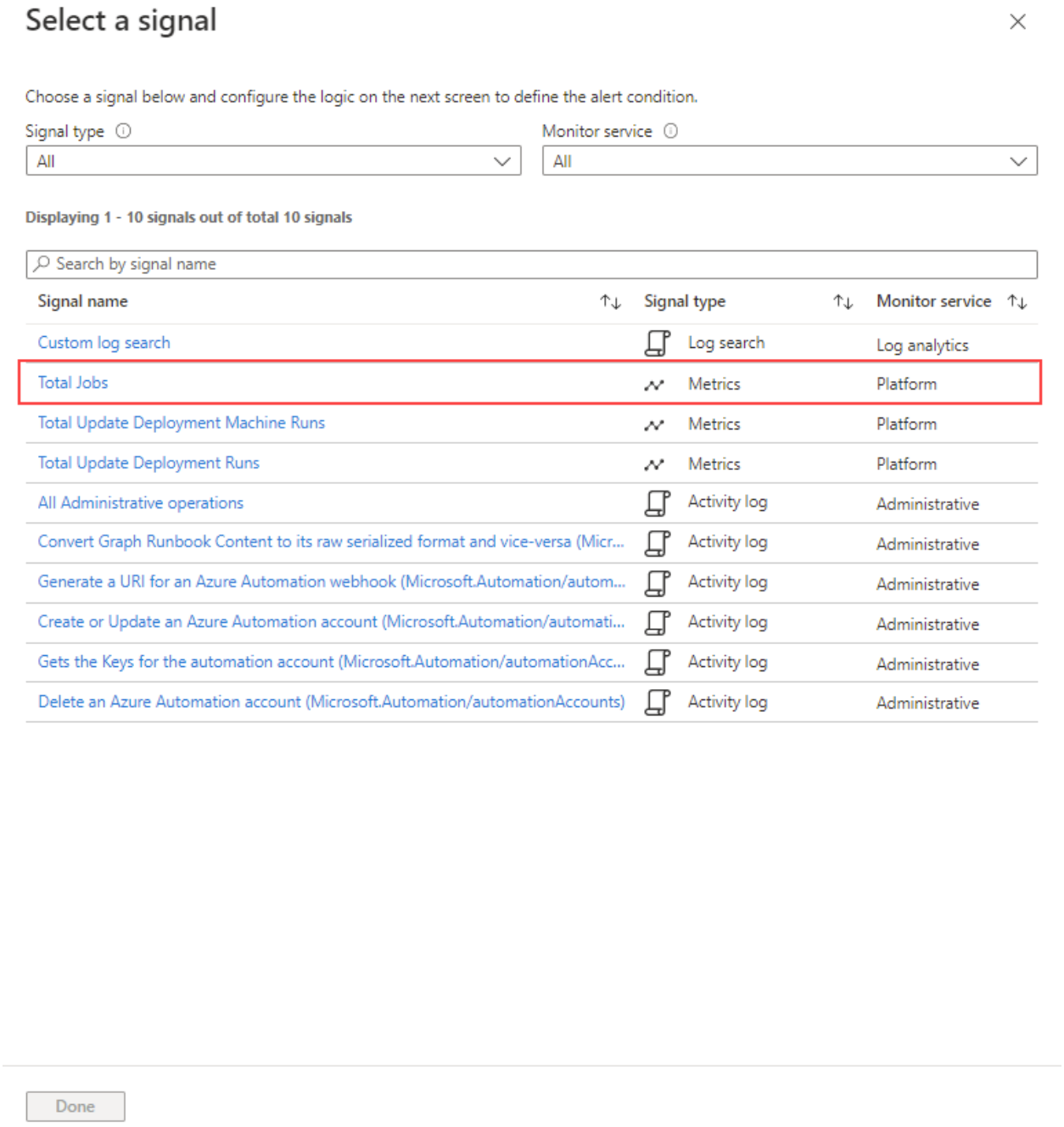
Alerts

You can also create metric alerts to monitor the execution of your runbooks.

- Alerts allow you to define conditions to monitor and actions to take when those conditions are met.

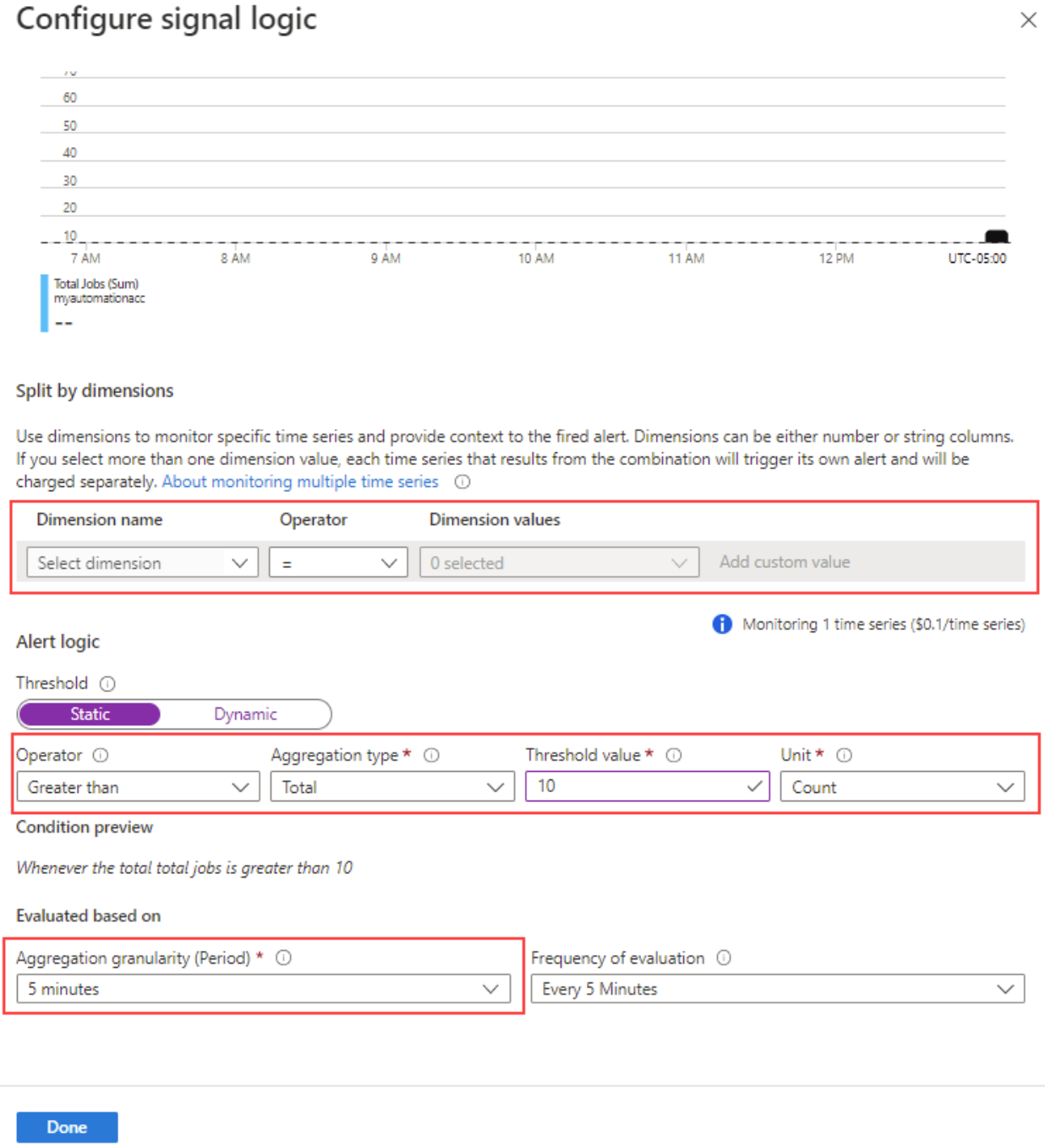


When you select Create alert rule, the Select a signal slide-out opens on the right side of your screen.
Next, select a signal that is most appropriate for your scenario.
For this example, select Total Jobs.



In the Configure signal logic slide-out, select Static for the Threshold property.

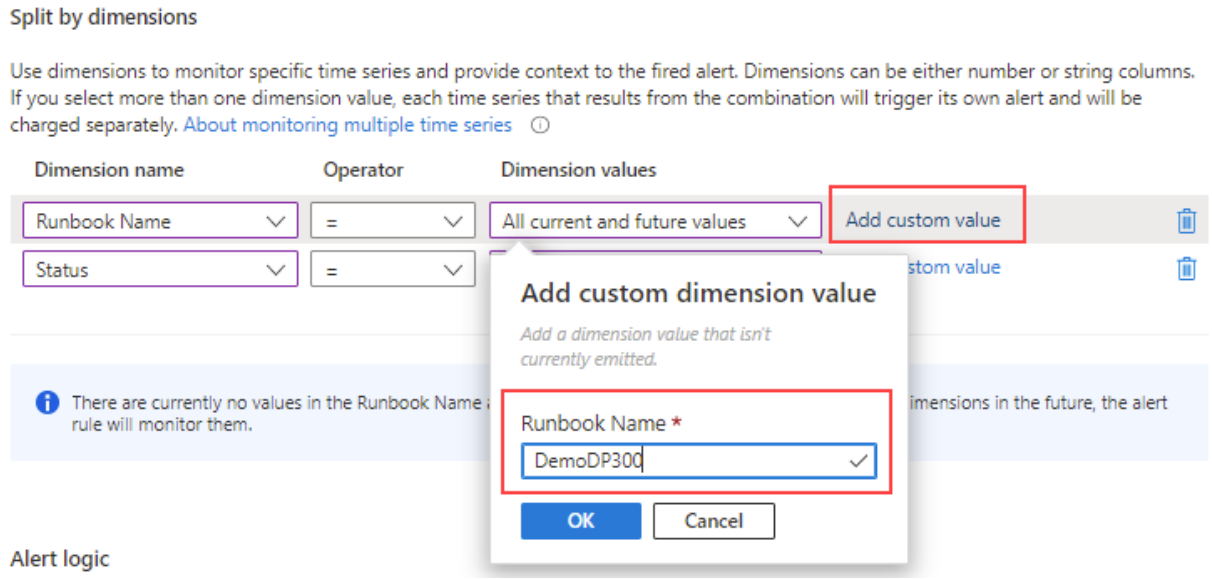
- Then, set the Operator property to Greater than, and the Aggregation type to Total.
- Enter 10 for the Threshold value.



Alternatively, you can specify a dimension.

- For example, you can define that an alert rule will only trigger for a specific runbook and status.

If you don't specify a dimension, no filter is applied.

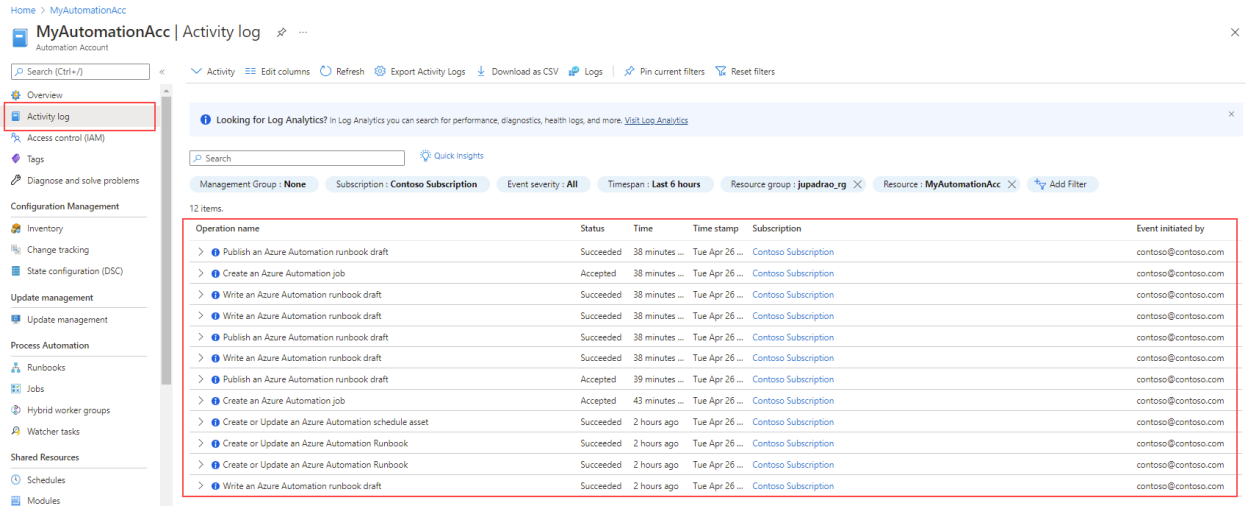


- Next, configure an action group.
- An action group is a collection of actions that you can use across multiple alerts, including email notifications, runbooks, webhooks, and more.

  [NOTE] You can combine Azure Automation with Azure Alerts action groups to initiate an Automation runbook when an alert is raised.

Activity log

- In Azure Automation, runbooks are executed and details are collected in an activity log.



- In the following image, you can retrieve runbook details, such as the person or account that started a runbook.
- As an alternative, the following PowerShell example provides the last user to run the specified runbook.

PowerShell

```
$rgName = 'MyResourceGroup'
$accountName = 'MyAutomationAccount'
$runbookName = 'MyRunbook'
$startTime = (Get-Date).AddDays(-1)
```

```
$params = @{
ResourceGroupName = $rgName
StartTime = $startTime
}
$JobActivityLogs = (Get-AzLog @params).Where( { $_.Authorization. Action -eq 'Microsoft.
Automation/automationAccounts/jobs/write' })
```

```
$JobInfo = @{}
```

foreach ($log in $JobActivityLogs) {

```
# Get job resource
$JobResource = Get-AzResource -ResourceId $log. ResourceId

if ($null -eq $JobInfo[$log. SubmissionTimestamp] -and $JobResource. Properties. Runbook. Name -eq $runbookName) {
# Get runbook
$jobParams = @{
ResourceGroupName = $rgName
AutomationAccountName = $accountName
Id = $JobResource. Properties. JobId
}
$Runbook = Get-AzAutomationJob @jobParams | Where-Object RunbookName -EQ $runbookName

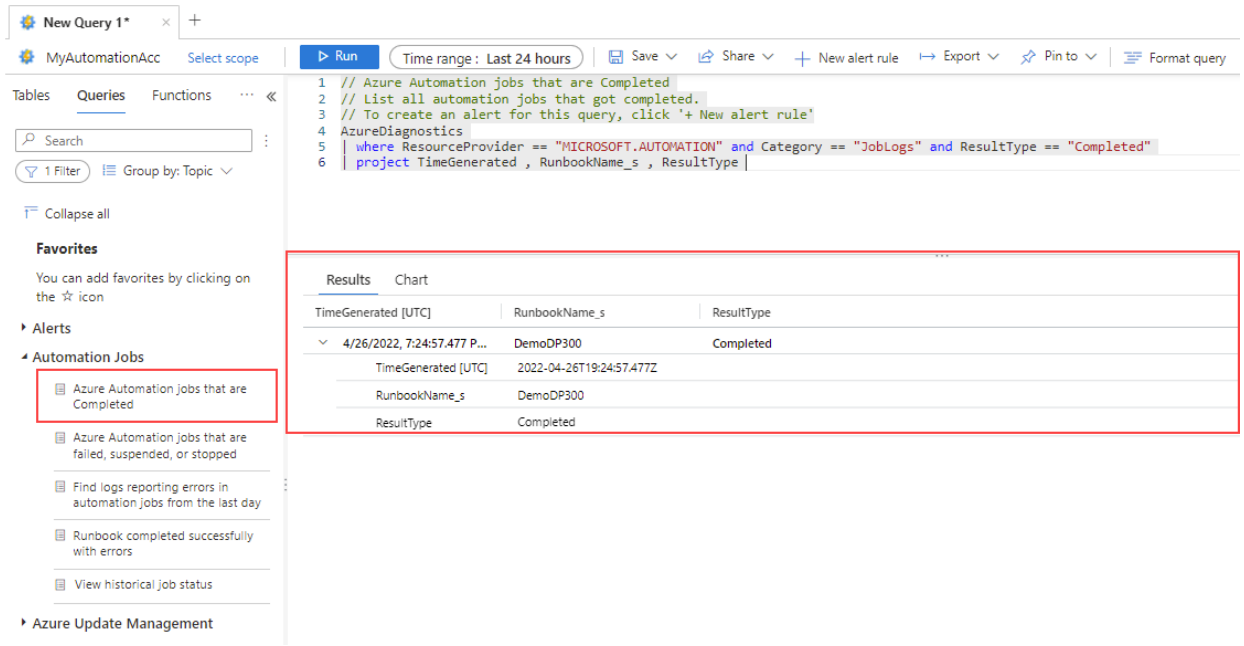
# Add job information to hashtable
$JobInfo. Add($log. SubmissionTimestamp, @($Runbook. RunbookName, $Log. Caller, $JobResource. Properties.jobId))
}
}
$JobInfo. GetEnumerator() | Sort-Object Key -Descending | Select-Object -First 1
```

Log Analytics

- Azure Automation can send runbook job status and job streams to your Log Analytics workspace.
- This method doesn't require workspace linking and is independent.
- Azure Monitor logs integrated with Automation account, enables you to:
- **View the status of your Automation jobs**
- Write advanced queries across your job workflow
- Trigger an email or alert based on your runbook job status

  Correlate data from multiple Automation jobs

  To run queries, select Logs from the Monitoring section of your automation account main blade.
- The Azure portal provides a few query templates so you can get started.
- As we can see below, we used an existing Kusto query template to list all completed jobs in the automation account.



- The ability to forward Azure Automation diagnostic logs to a Log Analytics workspace is an important feature that helps you monitor the health of your runbooks.

- Note
- Before you start using Log Analytics to query Automation jobs data, you must configure Diagnostic settings for your Automation Account.

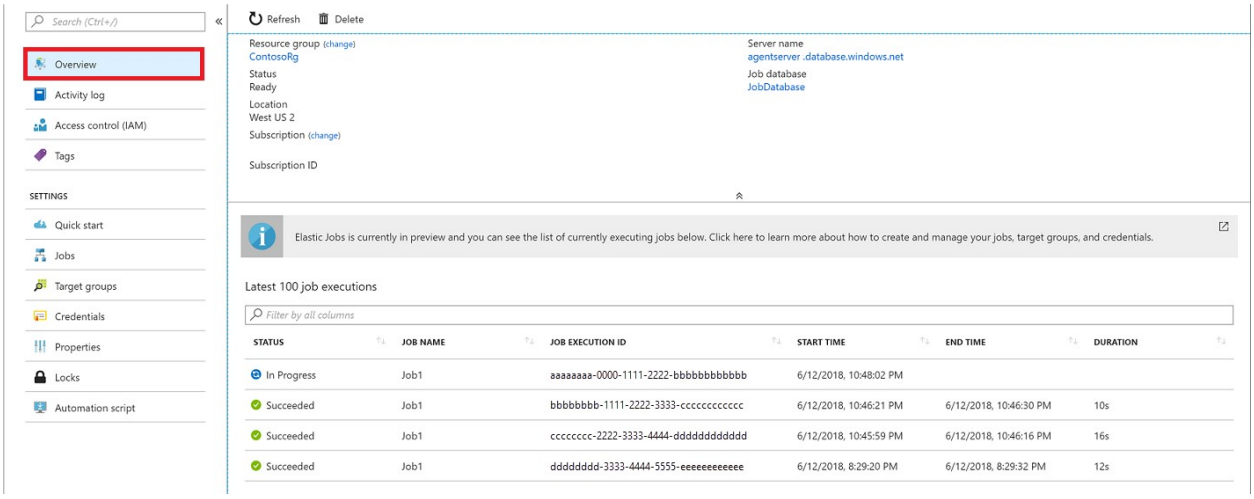
Monitor elastic jobs

With elastic jobs, job executions are logged, and any failures are automatically retried.

- The automatic retry feature provides services more resilient to transient failures.
- You can monitor elastic jobs execution through Azure portal, PowerShell, and T-SQL.

Azure portal

To view the history of jobs execution, select Overview for your Elastic Job agent main blade.



PowerShell

- The following code snippets get job execution details.

PowerShell

```
# get the latest 5 executions run
$jobAgent | Get-AzSqlElasticJobExecution -Count 5
```

```
# get the job step execution details
$jobExecution | Get-AzSqlElasticJobStepExecution
```

```
# get the job target execution details
$jobExecution | Get-AzSqlElasticJobTargetExecution -Count 2
```

T-SQL

- The following example shows how to view execution status details for all jobs.

Create a job agent and connect to the job database, then run the following command:

SQL

```
--View all execution status for all jobs
SELECT *
FROM jobs.job_executions
ORDER BY start_time DESC;
```

```
--View all execution statuses for job named 'MyJob'
SELECT * FROM jobs.job_executions
WHERE job_name = 'MyJob'
ORDER BY start_time DESC;
```

```
-- View all active executions
SELECT *
```

```
FROM jobs.job_executions
WHERE is_active = 1
ORDER BY start_time DESC;
```

Exercise: Deploy an automation runbook to automatically rebuild indexes

In this exercise, you'll learn how to create an automation runbook to automatically rebuild indexes.

Note

- To complete this exercise, you'll need an Azure subscription.
- Launch the exercise and follow the instructions.

Launch Exercise

Summary

- **1 minute**
- In this module you learned about various ways to automate common tasks in Azure and within SQL Server.
- One of the benefits of cloud computing is that a framework for robust automation, monitoring, and tooling is part of the platform.
- In the on-premises world, a complex set of tools would need to be assembled just to match the built-in functionality available with Azure metrics, policy, and automation.
- Additionally, SQL Server provides an automation framework using SQL Agent, and a robust monitoring solution via Extended Events.
- You can take advantage of both Azure Automation and Azure elastic jobs to automate the management of your Azure resources.